

Informe de Lógica de Negocio y Flujos Funcionales

Informe de Lógica de Negocio y Flujos Funcionales

Análisis: Cumplimiento de Reglas de Negocio en el Código

Este informe detalla cómo están implementadas actualmente las reglas de negocio (procesos, flujos, cualificación) en el código fuente, cruzando las alertas funcionales de la auditoría con la realidad técnica.

1. Flujo de Seguimiento de Varios Días (Orquestador Secuencial)

La Regla de Negocio Esperada:

El sistema debe ejecutar una secuencia de seguimiento escalonada. Ejemplo: Llamar el Día 1, enviar WhatsApp el Día 2, volver a llamar el Día 3, etc., respetando zonas horarias y pausándose si el usuario responde o se cualifica.

Realidad en el Código (`lib/core/orchestrator.ts`):

-  **Lo que SÍ se cumple:**
 - El motor lee la configuración `sequence` correctamente (Ej: [{step: 1, action: 'call'}, {step: 2, action: 'whatsapp'}, ...]).
 - Existen salvaguardas funcionales (`stopNow`, `shouldStopSequence`) que detienen la secuencia si el lead es "Cualificado", "Agendado" o "Descartado".
 - Hay un comprobador de inactividad: Si el lead ha escrito por WhatsApp en los últimos 20 minutos, la secuencia se pausa para no interrumpir al agente humano.
 - El Guardia de Zona Horaria (`buildComplianceDecision`) funciona: Si es de madrugada para el lead, salta al paso de WhatsApp o encola la llamada.

Dónde está el fallo (El Riesgo 4 de la auditoría):

- Al final de la función `executeSequenceStep`, el código dice: `await this.queueStep(..., delayMs)`. El sistema confía en una cola BullMQ (`lead-sequence-queue.ts`) para ejecutar el paso 2 y 3.
- **Fallo Silencioso:** Si el Worker de BullMQ no está corriendo en el servidor (un servicio de fondo separado), los trabajos se encolan pero nunca se ejecutan. Esto provoca exactamente el síntoma reportado: el Paso 1 se hace en vivo, pero los pasos diferidos (horas después) se quedan congelados en la cola inactiva.

2. Criterios y Estados de Cualificación (El Motor de IA)

La Regla de Negocio Esperada:

El sistema de IA debe evaluar a los leads con criterios específicos (ej. 2 años de experiencia) y marcar de

forma unificada si un lead es apto. Tras cualificarse, debe asignarse a un asesor (Virginia) y pasar a estado "Agendamiento".

Realidad en el Código:

-  **Lo que SÍ se cumple:**
 - La función `handleLeadQualification()` hace exactamente el flujo pactado en el CRM (Zoho): cambia el Owner al ID 781577000032471016 (Virginia), añade la etiqueta `Cualificado_Virginia`, cambia la etapa a `SCHEDULING` y crea una Tarea de Seguimiento en el CRM.
 - El sistema extrae dinámicamente las reglas de cualificación del curso desde la base de datos (`qualification_rules`) y se las pasa al Prompt del agente en `executeAIAgentStep`.

Dónde está el fallo (El Riesgo 5 de la auditoría):

- **Inconsistencia de umbrales:** Dado que las reglas de cualificación se inyectan como texto en el prompt (`qualification_rules`), es altamente probable que el texto guardado en la base de datos diga "3 años" en lugar de "2 años", o que el LLM esté alucinando el umbral. (poco probable pero hay posibilidad)
- **Inconsistencia de Estados:** A lo largo del código, el estado se guarda de diferentes formas. En Supabase usa `current_stage: "SCHEDULING"`, en los metadatos usa `qualified: true`, y en Zoho usa la etiqueta `Cualificado_Virginia`. No hay un Enum estricto en TypeScript que obligue a usar un solo formato estándar, lo que provoca la desincronización mencionada en el informe.

3. Asignación de Asesores (Round Robin)

La Regla de Negocio Esperada:

Los leads cualificados deben distribuirse de manera equitativa entre los asesores de ventas disponibles, y se debe comprobar su disponibilidad en el calendario.

Realidad en el Código (`lib/actions/scheduling.ts`):

-  **Lo que SÍ se cumple:**
 - Existe toda la estructura para el Round Robin. La función `handleLeadQualification` llama a `getNextAdvisor()` para elegir al siguiente vendedor.
 - El sistema cruza los horarios (`availability_slots`) con las citas ya agendadas (`appointments`) para evitar solapamientos (`checkAvailability`).
 - Considera el huso horario al programar recordatorios automáticos (restando `leadTimeMinutes`).

4. Control de Costes y Límites Diarios

La Regla de Negocio Esperada:

El sistema no debe exceder un límite de gasto diario para proteger la cuenta.

Realidad en el Código:

-  Lo que Sí se cumple:
 - El Orchestrator implementa un Circuit Breaker (Cortacircuitos): Antes de cada paso, verifica `current_daily_spend >= daily_spend_limit`. Si se supera, cancela la acción con el log "Daily Spend Limit Reached".
 - Implementa "Pacing" (Ritmo): Verifica si se están enviando demasiados mensajes en poco tiempo (`checkPacing`) y los aplaza para evitar baneos de WhatsApp o saturar la centralita.

5. Escalado a Humanos (Handoff)

La Regla de Negocio Esperada:

Si el IA falla en la llamada repetidas veces, o el lead resulta ser un número no válido, debe informarse al CRM o intentar por WhatsApp.

Realidad en el Código:

-  Lo que Sí se cumple:
 - Si la llamada devuelve un error de "invalid_destination" (teléfono falso), el sistema activa automáticamente un BLUEPRINT en Zoho etiquetando al lead como "Anulado automáticamente por IA - Número Inválido".
 - Si un lead es cualificado (SCHEDULING) pero al final de la llamada no agendó (`outcome !== "BOOKED"`), el sistema activa la regla `handoff_on_qualified_not_booked` (si está habilitada en la config) para derivarlo a un agente humano.

Conclusión sobre la Lógica de Negocio

La arquitectura técnica de las reglas de negocio **es correcta y robusta en su diseño** (contempla fallbacks, round robin, límites de gasto, timezone guards).

Los problemas reportados (Riesgos 4 y 5) son fallos de ejecución infraestructural y de consistencia de datos, no de falta de desarrollo lógico:

- **1. El paso 2 y 3 no saltan:** Porque el proceso en segundo plano (BullMQ Worker) probablemente no está activo en el servidor de producción.

- **2. Cualifica mal:** Porque el texto que se le pasa al Prompt en `qualification_rules` no está alineado con lo que el cliente pidió, y al carecer de tipos estrictos para el estado "Cualificado", partes del dashboard no se enteran de que la IA ya lo aprobó.

Informe Maestro del Sistema

Informe Maestro del Sistema — Automatiza Formación Dashboard

Fecha de auditoría: 2026-05-21

1. Configuraciones (/dashboard/settings)

Feature	Estado	Detalle
Panel multi-tenant (ADMIN)	✓	Gestión completa de tenants
Configuración de WhatsApp	✓	Guardado en tenant.config
Configuración de AWS Bedrock	✓	Guardado en tenant.config
Configuración de Retell	✓	Guardado en tenant.config
Configuración de agentes por defecto	✓	Asocia agentes IA al tenant
Validación de credenciales	⚠	Solo verifica existencia, no validez

2. Orquestador (/dashboard/orchestrator)

Feature	Estado	Detalle
Constructor visual de Workflows	✓	Creación de flujos multi-paso
Tipos de pasos	✓	CALL, WHATSAPP, WAIT, AI_AGENT, LLM_TEXT
Motor de ejecución automático	✓	triggerOrchestratorForLead
A/B Testing de variantes	✓	Distribución porcentual configurable
Sub-Workflows (anidamiento)	✓	Un paso puede llamar otro workflow
Reglas de condición (IF/ELSE)	⚠	Estructura existe, editor limitado en UI

3. Agentes de Texto (/dashboard/agents)

Feature	Estado	Detalle
Creación y edición	✓	CRUD completo vía Server Actions

Feature	Estado	Detalle
System Prompt personalizable	✓	Almacenado en ai_agents.system_prompt
Selección de modelo LLM	✓	Campo model configurable
Variables de extracción	✓	Define datos a capturar
Knowledge Base asociada	✓	Vincula el agente al KB de AWS
Vista previa	⚠	Remite al Simulador

4. Agentes de Voz (/dashboard/voice-agents)

Feature	Estado	Detalle
Creación de agentes	✓	Integrado con Retell API
Configuración de voz	✓	Parámetros enviados a Retell
Script de conversación	✓	Prompt adaptado para voz

Feature	Estado	Detalle
Métricas del agente	✓	Visibles en el módulo Minutos
Clonación de voz personalizada	⚠	Depende de Retell externo

5. Knowledge Base (/dashboard/knowledge)

Feature	Estado	Detalle
Subida de documentos	✓	Carga a AWS S3 + indexación
Listado y gestión	✓	Visualización y eliminación
Sincronización con AWS Bedrock	✓	Re-indexación manual disponible
Búsqueda semántica	✓	Activa en cada respuesta
Soporte multi-formato	✗	Solo texto/PDF

6. Chatbot Web (/dashboard/web-chatbot)

Feature	Estado	Detalle
Widget embebible	✓	Script generado con IDs
Configuración visual	✓	Guardado en tenant.config
Agente IA respondiendo	✓	Conectado al agente de texto
Histórico de chats	✓	Almacenado en chat_messages
Soporte archivos adjuntos	✗	No implementado
Autenticación visitante	✗	Visitante siempre anónimo

7. Resumen de Leads (/dashboard)

Feature	Estado	Detalle
KPIs globales	✓	getKpiGenerales real
Embudo (funnel)	✓	Visualización completa

Feature	Estado	Detalle
Gráficos dinámicos	✓	ChartManager
KPIs personalizables	✓	Configuración en BD
Filtros avanzados	✓	FilterBar funcional
Comparación períodos	✗	No implementado

8. Conversaciones WhatsApp (/dashboard/conversaciones)

Feature	Estado	Detalle
Listado por lead	✓	Vista de hilos
Transcripción	✓	Mensajes cronológicos
Estado de cualificación	✓	Badge visual
Filtros	✓	Funcionales en servidor

Feature	Estado	Detalle
Búsqueda	⚠	Por nombre/teléfono, no contenido
Respuesta manual	✗	No contemplado

9. Calendario (/dashboard/calendar)

Feature	Estado	Detalle
Vista mensual	✓	react-big-calendar
Creación manual	✓	Modal funcional
Agendamiento IA	✓	Funcional
Propuesta horario	✓	Negociación automática
Sincronización Google	✗	No implementado
Recordatorios	✗	No implementado

10. Campañas (/dashboard/campanas)

Feature	Estado	Detalle
Creación	✓	Tabla campanas
Asignación agentes	✓	Selector
KPIs por campaña	✓	getKpiCampanas
Gráficos rendimiento	✓	getDynamicChartSeries
Tabla leads	✓	Paginada
Edición	⚠	Campos limitados
Pausa/Activación	⚠	Sin toggle en UI

11. Llamadas — Métricas (/dashboard/minutos)

Feature	Estado	Detalle
Total minutos	✓	Cálculo desde llamadas
Duración media	✓	Promedio real

Feature	Estado	Detalle
Tasa de éxito	✓	Cálculo getKpiMinutos
Gráficos	✓	getDynamicChartSeries
KPIs personalizables	✓	SummaryManager
Desglose agente	⚠	Global, no individual

12. WhatsApp — Métricas (/dashboard/whatsapp)

Feature	Estado	Detalle
Total conversaciones	✓	Hero metric
Tasa agendamiento	✓	Cruce con appointments
Tasa cualificación	✓	Cruce con lead_cualificacion
Gráficos	✓	Configurables

Feature	Estado	Detalle
KPIs personalizados	✓	SummaryManager
Filtro fechas	✓	Fecha último mensaje

13. Historial (/dashboard/historial)

Feature	Estado	Detalle
Tabla maestra	✓	Paginación servidor
Filtros combinados	✓	Supabase query
Búsqueda	✓	Nombre/teléfono
Timeline omnicanal	✓	Trazabilidad completa
Gestor columnas	✓	dnd-kit
Grabaciones	✗	Comentado
Exportación	✗	No implementado

14. Simulador (/dashboard/simulator)

Feature	Estado	Detalle
Chat interactivo	✓	Test variables
Monitor variables	✓	JSON real time
Selección agente	✓	Lista del tenant
Indicador estado	✓	Panel lateral
Persistencia	✗	Sandbox (se borra)
Simulación voz	✗	Solo texto

15. Playground (/dashboard/playground)

Feature	Estado	Detalle
Diagnóstico salud	✓	Verifica tokens/KB
Selección workflow	✓	Datos reales

Feature	Estado	Detalle
Consola en vivo	✓	Logs orquestador
Historial sistema	✓	system_logs
Modo Dry-Run	✓	Operativo
Traducción errores	✓	translateError()

16. Auditoría de Logs (/dashboard/logs)

Feature	Estado	Detalle
Eventos tiempo real	✓	Auto-refresh
Filtro resultado	✓	Cliente
Búsqueda	✓	Cliente
Inspector JSON	✓	Visor
Panel error	✓	Alert

Feature	Estado	Detalle
Botón Copiar JSON	✗	Sin función
Búsqueda histórica	✗	Limitado a 50
Paginación	✗	No implementado

Resumen Ejecutivo

Módulo	✅ Funcional	⚠ Limitaciones	✗ No Implementado
Configuraciones	5	1	0
Orquestador	5	1	0
Agentes de Texto	5	1	0
Agentes de Voz	4	1	0
Knowledge Base	4	0	1

Módulo	✅ Funcional	⚠ Limitaciones	❌ No Implementado
Chatbot Web	4	0	2
Resumen de Leads	5	0	1
Conversaciones WhatsApp	4	1	1
Calendario	4	0	2
Campañas	5	2	0
Llamadas (Métricas)	5	1	0
WhatsApp (Métricas)	6	0	0
Historial	5	0	2
Simulador	4	0	2
Playground	6	0	0

Módulo	✅ Funcional	⚠ Limitaciones	❌ No Implementado
Auditoría Logs	5	0	3
TOTAL	80	8	14

Sección de Configuraciones

Reporte Detallado: Sección de Configuraciones

admin/cliente

Elaborado por: Renzo

Sección Analizada: Configuración de Clientes y Entornos (Settings / Dashboard)

Este documento detalla exhaustivamente cada función, botón, y campo existente en la sección de Configuraciones de la plataforma. El objetivo es que quede totalmente claro qué hace cada cosa, cómo debería funcionar de manera ideal y qué errores o cosas faltan actualmente. Está redactado para ser entendido fácilmente y sin términos excesivamente técnicos.

1. Pantalla Principal: Lista de Clientes (page.tsx)

Esta es la vista donde se ven todos los clientes y administradores activos en el sistema.

Botón: "Nuevo Cliente"

- **Lo que debería hacer:** Al hacer clic, debe abrir un panel limpio y amigable para que registres un nuevo usuario (sea cliente o administrador de la plataforma) e ingreses sus credenciales, además de las conexiones base.
- **Lo que hace actualmente:** Abre efectivamente el formulario.
- **Errores / Faltantes detectados:**
 - Si ocurre un fallo en el servidor al intentar crear, en lugar de mostrar un aviso amigable y claro (ej. "El correo ya existe"), muestra una alerta genérica y fea del navegador ("Error desconocido" o "Error crítico").

Tabla de Clientes Activos

- **Lo que debería hacer:** Mostrar todos los clientes creados con su información básica (Nombre, si usan servidor interno o propio, su correo y su rol).
- **Lo que hace actualmente:** Muestra la lista y un ícono indicando si es Administrador o Cliente.
- **Errores / Faltantes detectados:**
 - No tiene buscador ni filtros. Si llegas a tener 50 clientes, tendrás que buscar visualmente uno por uno.
 - Si un cliente usa su propio servidor (API del Cliente), el texto de la URL sale "cortado" para que no ocupe mucho espacio, pero no permite hacer clic en él o copiar la URL rápidamente.

Acciones de la Tabla: Botones "Lápiz" (Editar) y "Basurero" (Eliminar)

- **Lo que debería hacer (Eliminar):** Al darle clic al basurero, debe pedir una confirmación de seguridad robusta (por ejemplo, escribir "ELIMINAR") para evitar borrar un cliente completo por un clic accidental, ya que esto destruiría su acceso.
- **Lo que hace actualmente (Eliminar):** Solo lanza un pequeño cuadro de aviso nativo del navegador "¿Estás seguro...?". Si alguien da "Enter" sin querer, el cliente se borra y no hay botón de "Deshacer". **(Crítico)**
- **Lo que debería hacer (Editar):** Al darle al lápiz, debería abrir un panel lateral o ventana para editar al cliente de forma ordenada.
- **Lo que hace actualmente (Editar):** Abre la edición "encima" de la misma fila, volviendo toda esa parte azul y mostrando todos los campos apretados.

2. Formulario de Cliente / Edición

Cuando creas o editas un cliente, aparecen diferentes bloques de información que debes rellenar.

Selector de Rol: Botón "Admin" / "Cliente"

- **Lo que debería hacer:** Alternar entre un perfil de administrador central (que no necesita integraciones extra) y un perfil de cliente (que sí las necesita).
- **Lo que hace actualmente:** Al hacer clic en "Admin", oculta correctamente todos los bloques de integraciones, KPI y telefonía porque el admin no necesita eso. Si haces clic en "Cliente", vuelve a mostrarlos. Funciona bien, pero no es intuitivo que ese pequeño botón en la parte superior cambie toda la configuración de abajo.

Bloque: Infraestructura de Datos (API Interna vs API Externa)

- **Lo que debería hacer:** Permite escoger si el cliente usará tu misma base de datos (Interna) o si tiene la suya propia (Externa/Supabase).
- **Lo que hace actualmente:** Al elegir "Externa", despliega dos campos nuevos: URL y Key.
- **Errores / Faltantes detectados:**
 - No hay ningún botón de "Probar Conexión". El usuario puede pegar la "Key" con un espacio en blanco al final por accidente, guardarlo, y no se enterará de que la conexión falló hasta que todo el sistema de ese cliente se caiga.

Bloque: Configuración del Panel (Título y Headers)

- **Lo que debería hacer:** Permitir cambiar el título que ve el cliente cuando entra y los nombres de las columnas (Headers) de sus datos.
- **Lo que hace actualmente:** Tiene dos cajas de texto simples.
- **Errores / Faltantes detectados:**
 - El sistema requiere que los 'Headers' se escriban separados por comas (ejemplo: Nombre, Teléfono, Correo). Si el usuario pone comillas o caracteres raros, el sistema no te avisa, pero luego las columnas de datos se verán rotas para el cliente final.

3. Creador de Indicadores Visuales (KpiBuilder.tsx)

Esta sección permite crear los "Cuadraditos de colores" (Módulos de panel) con estadísticas numéricas (KPIs) que ve el cliente en su pantalla principal.

Botón: "Agregar KPI" y Bloques de KPI

- **Lo que debería hacer:** Al presionarlo, crear una nueva caja para configurar un indicador (ej. Total de Ventas, Llamadas perdidas, etc.).
- **Lo que hace actualmente:** Agrega el bloque correctamente y permite elegir un color, título, tamaño en pantalla, la operación matemática (sumar, contar, promedio) y filtros opcionales.
- **Errores / Faltantes detectados:**
- **Los campos son "texto libre" peligrosos:** El campo de "Columna a calcular" requiere que escribas exactamente el nombre de la columna técnica en la base de datos (Ej: total_mins). Si escribes Total de minutos (con espacios) el panel del cliente explotará mostrando un error o ceros, porque no hay validación de que lo que se escribe sea válido.
- **Filtros limitados e inestables:** El bloque "Condición Opcional" dice "Si la columna X es igual a Y". En "Y", si la regla es de verdadero o falso, el usuario tiene que saber que debe escribir exactamente true o false. Si escribe verdadero o VERDADERO, el sistema fallará sin avisar.

4. Administrador de Integraciones (IntegrationsManager.tsx)

Aquí se conectan aplicaciones de terceros como Retell, WhatsApp y Google Sheets.

Integración: Retell AI (Voz) y Ultravox AI

- **Lo que debería hacer:** Pedir las "API Keys" (Llaves maestras de estos servicios) y verificar que sirvan.
- **Lo que hace actualmente:** Tiene un campo de contraseña para poner la Key y un botón "Sincronizar Recursos" en Retell.
- **Errores / Faltantes detectados:**
 - Ultravox NO tiene botón de "Sincronizar" ni de "Probar". Si la llave está mal, no hay forma de saberlo desde aquí.

Integración: Webhook CRM (Conexiones con Zoho/HubSpot)

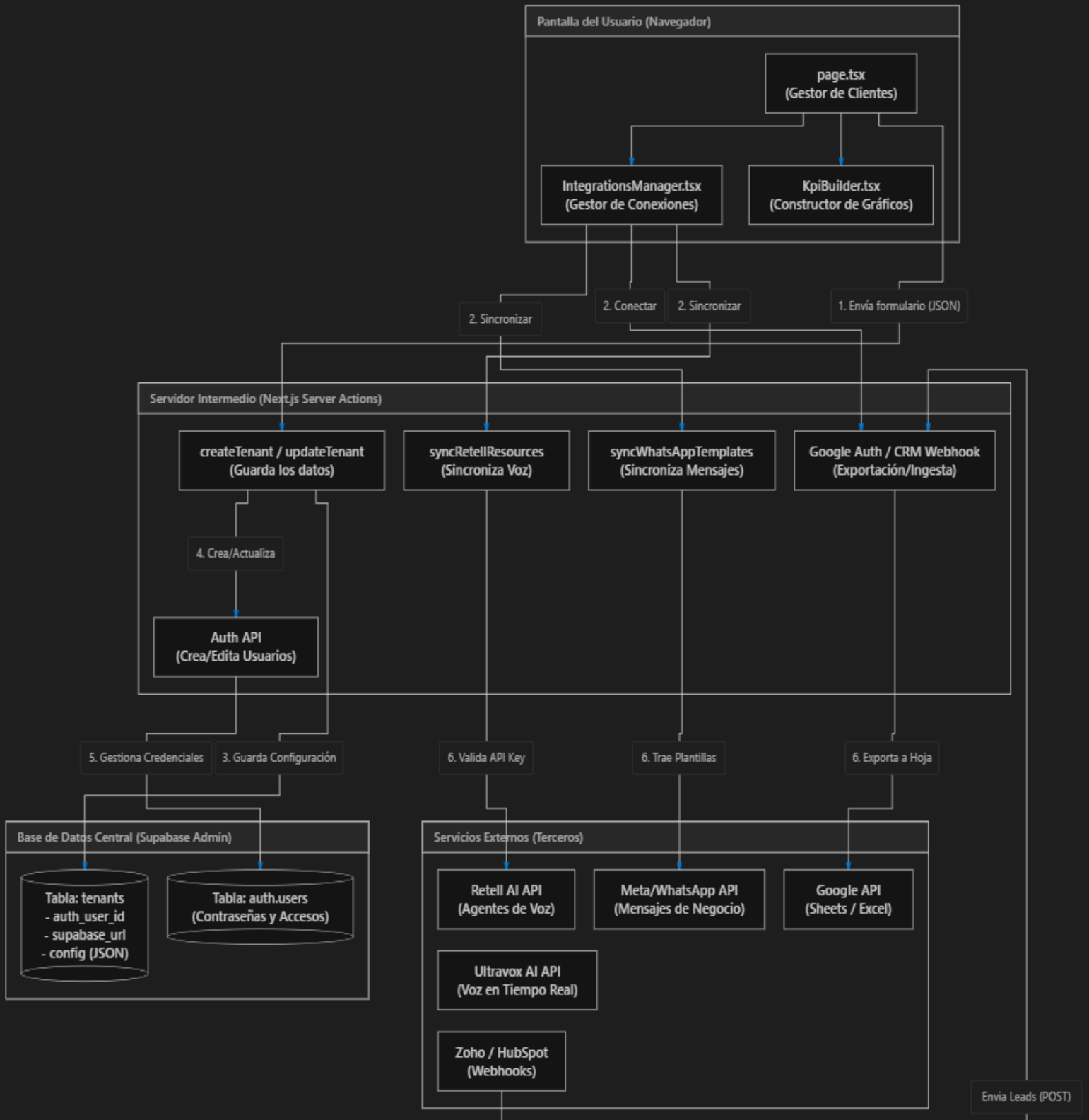
- **Lo que debería hacer:** Proveer un enlace único al cliente para que sus bases de datos externas envíen contactos al instante y ofrecer un botón de copiar y pegar fácil.
- **Lo que hace actualmente:** Muestra la URL y la llave "x-tenant-id" (el código del cliente).
- **Errores / Faltantes detectados:**
- **Falta botón de "Copiar":** El usuario tiene que seleccionar el texto con el mouse. Parece un detalle pequeño, pero los clientes se confunden y a veces copian espacios adicionales.
- Si abres el formulario para "Nuevo Cliente" (donde el cliente aún no se ha guardado ni tiene ID propio), el campo ID muestra el texto ID_DEL_CLIENTE. Si alguien copia eso y lo pega en Zoho, jamás llegarán los datos.

Integración: WhatsApp API (Meta)

- **Lo que debería hacer:** Permitir conectar la línea de WhatsApp del negocio y descargar automáticamente los mensajes pre-aprobados (Plantillas).
- **Lo que hace actualmente:** Pide el Access Token, ID de teléfono, ID de WABA y muestra la URL del webhook. El botón "Sincronizar Plantillas" funciona y te trae un resumen de las plantillas existentes.
- **Errores / Faltantes detectados:**
- **Límite Visual:** Si tienes más de 6 plantillas de WhatsApp, el panel muestra un cuadrado que dice "+ X más...". ¡No hay forma de ver, desde esta pantalla, cuáles son esas plantillas extra! El cliente no puede ver la lista completa.
- **Alerta confusa:** Si el cliente da click a "Sincronizar Plantillas" mientras está creando un "Nuevo Cliente" (antes de darle al botón general "Desplegar Cliente"), el sistema tira una alerta nativa ruda que dice "Primero debes guardar el cliente antes de sincronizar". El botón de sincronizar debería estar gris (deshabilitado) hasta que el cliente sea guardado por primera vez.

Integración: Google Sheets (Excel en línea)

- **Lo que debería hacer:** Permitir vincular el correo de Google y exportar clientes a un Excel en automático.
- **Lo que hace actualmente:** Tiene un botón para "Conectar Google" que te lleva al inicio de sesión de Google. Luego pide el "Spreadsheet ID" (El código de la página del Excel) y el nombre de la pestaña (Hoja1, Leads, etc.).
- **Errores / Faltantes detectados:**
- Igual que WhatsApp, si intentas conectar Google sin haber guardado al "Nuevo Cliente" primero, lanza la alerta.
- **No hay botón para desconectar:** Una vez que conectas tu cuenta de Google, el botón cambia a "Reconectar Google". Si por algún motivo el cliente quiere desenlazar la cuenta por seguridad y revocar permisos de Google, la interfaz no cuenta con un botón "Desconectar" o "Remover conexión".
- **Spreadsheet ID Manual:** Tienes que pedirle al cliente que vaya a la URL (enlace web) de su Excel, seleccione con el mouse un código larguísimo entre un montón de barras (/d/ESTO/edit), y lo pegue allí. Es extremadamente propenso a errores humanos, si pegan mal un número, el sistema intentará guardar información en un Excel inexistente.



Módulo del Orquestador

Reporte Detallado: Módulo del Orquestador (Onboarding / Flujos)

Elaborado por: Renzo

Sección Analizada: Constructor Visual de Flujos (Orquestador v5.0)

Este documento detalla exhaustivamente cómo funciona el motor del Orquestador (ahora integrado en la sección de Onboarding). Explicaré para qué sirve cada "nodo" (las cajitas del diagrama), a dónde va la información por detrás, y cuáles son los fallos críticos actuales.

1. Funcionamiento General (El "Lienzo")

El orquestador permite crear secuencias automáticas arrastrando cajitas y conectándolas con flechas.

¿A qué tabla de la Base de Datos va esto?

El sistema tiene un problema de "doble personalidad" (está a medio migrar). Guarda información en dos lugares distintos:

- **1. Tabla** `tenant_orchestrator_config`: Guarda el dibujo visual (`flow_graph` con todas las coordenadas x/y de las cajitas) y las reglas generales.
- **2. Tablas** `workflows` y `orchestration_rules`: Aquí es de donde el motor real (`orchestration.ts`) lee las reglas paso a paso para ejecutarlas de verdad.

ERROR CRÍTICO AL GUARDAR:

Al presionar el botón de "Guardar Configuración" en el orquestador, el sistema solo guarda el dibujo (las flechas y cajitas) pero olvida guardar las configuraciones de los nodos y del sistema en la base de datos (se obvia el objeto `config` en la función de guardado). Esto significa que un usuario puede configurar los horarios o un mensaje, y al guardar, la base de datos nunca lo registra, quedándose solo con el esquema visual.

2. Nodos del Orquestador (Las Cajitas)

Aquí detallo qué hace cada bloque disponible y sus fallos.

Disparadores (Triggers)

Son las cajitas que inician el flujo automático.

- **Disparador (`LeadTriggerNode`)**
 - **Qué hace:** Despierta el sistema cuando entra un nuevo Lead (contacto) desde una campaña específica o cualquier origen.

- **Faltante/Error:** Es un campo de texto libre. Si el usuario escribe "Facebook" pero los datos entran como "facebook" (minúscula), el flujo no se disparará.
- **Webhook de Entrada (WebhookNode)**
 - **Qué hace:** Genera una URL (ej. midominio.com/webhook) para que sistemas externos (como Zapier o un CRM) le "avisen" al orquestador que inicie.

Nodos de Condición (Los "Semáforos")

Deciden por qué camino debe ir el contacto.

- **Condición Horaria (TimeConditionNode)**
 - **Qué hace:** Verifica si en ese momento es un día laborable (ej. Lunes a Viernes de 09:00 a 20:00). Se adapta automáticamente al país del contacto según el prefijo de su número telefónico (ej. +34 para España, +52 para México).
 - **Faltante:** No permite agregar días feriados o festivos. Si es Navidad, el sistema seguirá llamando porque es un día laborable estándar. No valida si pones "25:00" como hora.
- **Condición If/Else (ConditionNode)**
 - **Qué hace:** Revisa variables matemáticas lógicas. Por ejemplo: ¿El cliente contestó la llamada? (Sí/No).
 - **Faltante/Error:** Obliga al usuario a escribir código técnico como `{{call.answered}} == true`. Un cliente normal no sabe escribir esto y si se equivoca por un espacio, el flujo se rompe.

Nodos de Acción (Los "Ejecutores")

Son los que hacen el trabajo pesado.

- **Llamada IA (VoiceCallNode)**
 - **Qué hace:** Dispara una llamada usando el robot de voz configurado (Retell AI o Ultravox).
 - **Fallo:** Si seleccionas un Agente que fue borrado en la plataforma de Retell, la cajita no avisa que el agente ya no existe y la llamada fallará silenciosamente.
- **WhatsApp (WhatsAppNode)**
 - **Qué hace:** Envía un mensaje de Meta usando una plantilla pre-aprobada.
 - **Fallo:** Las variables de la plantilla (ej. el nombre del cliente) se tienen que escribir en formato JSON (código puro). Es muy poco amigable para un usuario normal.
- **Agente de Texto (TextAgentNode)**
 - **Qué hace:** Usa inteligencia artificial para chatear o procesar textos.
- **Petición API (APINode)**
 - **Qué hace:** Le "avisa" a otra plataforma (ej. Zoho) que pasó algo, enviando datos silenciosos (POST).
 - **Fallo:** No hay forma de ver la respuesta de esa API si da un error, lo que hace que sea muy difícil saber por qué Zoho no recibió la información.

- **Espera (DelayNode)**
 - **Qué hace:** Pausa el flujo por X horas (ej. "esperar 24 horas antes del siguiente WhatsApp").
- **Bucle de Contacto (RetrySequenceNode) [Nuevo v5.1]**
 - **Qué hace:** Automáticamente intenta llamar o enviar WhatsApp varias veces (ej. 5 intentos cada 27 horas) hasta que el cliente conteste.

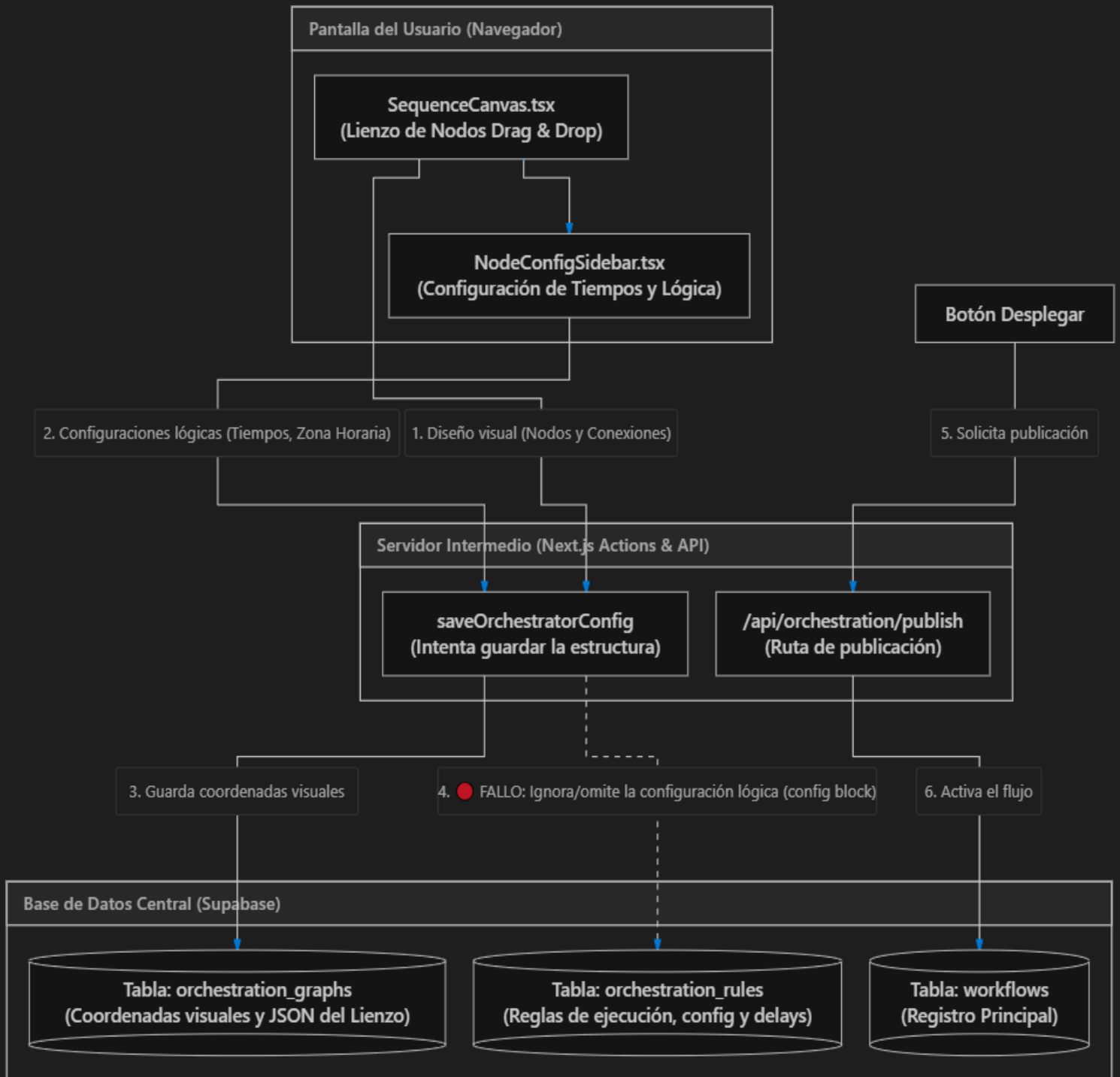


Nodos de Finalización

- **Vincular Flujo (SubWorkflowNode)**
 - **Qué hace:** Al terminar un flujo, envía al usuario a otro flujo distinto.
- **Fin de Flujo (EndNode)**
 - **Qué hace:** Cierra la caja y marca al contacto como finalizado en esa etapa.

3. Resumen Técnico del Estado Actual

- **1. ¿Funciona?** Visualmente sí, es muy atractivo. Mecánicamente, está roto en el proceso de guardado. El saveOrchestratorConfig pierde la configuración real y las variables del usuario, por lo que el motor no sabe qué ejecutar en la vida real.
- **2. Deuda Técnica:** El sistema intenta leer de una tabla nueva (workflows), pero el lienzo visual guarda en una tabla vieja (tenant_orchestrator_config). Hasta que no se unifiquen, el orquestador visual y el motor real de llamadas no van a coincidir.



Módulo de Agentes de Texto IA

Reporte Detallado: Módulo de Agentes de Texto IA (Cerebro, Memoria e Integración)

Elaborado por: Renzo

Sección Analizada: Consola de Orquestación de Agentes de Texto (/dashboard/agents)

Este documento detalla el estado, funcionamiento técnico y fallos del constructor de Agentes de Texto IA (el cerebro asíncrono que conversa con los clientes, por ejemplo, vía WhatsApp).

1. Estructura del Agente (La Pantalla y sus Pestañas)

El sistema organiza cada Agente IA en base a un "Agente Maestro" (ej. Virginia) que tiene tres grandes pestañas de configuración: Cerebro (Brain), Inactividad y CRM Sync.

¿A qué tablas de la Base de Datos va esta sección?

- **1. Tabla ai_agents:** Registra el Agente Maestro (ID, Nombre, Descripción, si está Activo y su tipo). Está enlazado a la cuenta del cliente (tenant_id).
- **2. Tabla ai_agent_variants:** Guarda la configuración de la "inteligencia" y los prompts (el ADN, el modelo de IA elegido, las variables que debe recordar, las reglas de inactividad, y la sincronización con el CRM).

2. Pestañas y Funcionalidades en Detalle



Pestaña 1: Cerebro (El Motor e Instrucciones)

Es el núcleo del pensamiento del robot.

- **Proveedor y Modelo de IA (Cerebro):**
 - **Qué hace:** Permite elegir qué "cerebro" ejecutará al agente. Soporta OpenAI (ChatGPT), Anthropic (Claude) y Google Gemini.
 - **⚠ FALLO / CONFUSIÓN DE NOMBRE DE MODELOS:**
En la interfaz se ofrecen opciones como GPT-4.1 (Omni) y GPT-4.1 Mini. Estos modelos no existen oficialmente en la API pública de OpenAI (las versiones oficiales son gpt-4o, gpt-4o-mini, etc.). Si un usuario los selecciona y el backend intenta llamarlos de forma literal sin un traductor interno, la inteligencia del robot fallará arrojando un error de API.
- **ADN del Agente (System Prompt):**
 - **Qué hace:** Es el cuadro de texto donde se le da el rol al robot (ej: "Eres un asesor educativo experto, tu meta es agendar una cita...").
- **Política de Finalización:**

- **Qué hace:** Le dice al robot cuándo considerar que la conversación terminó con éxito o fracaso (ej: "Cuando el usuario diga explícitamente que no le interesa").
- **Capturar Datos (Memoria del Agente):**
 - **Qué hace:** El robot escucha la conversación y guarda de forma automática campos clave del cliente en variables como {{USER_NAME}}, {{COURSE_NAME}}, etc. Permite asignarles tipos como Texto, Número, Booleano, Email o Teléfono.
 - **FALLO DE DISEÑO (Variables fantasma): no es crítico a su funcionalidad sino como aviso a considerar**
Si la base de datos está vacía, la interfaz simula y muestra por defecto varias etiquetas comunes (ej. USER_NAME, COURSE_NAME, QUALIFIED). Sin embargo, estas etiquetas son "fantasmas": no existen realmente en la base de datos hasta que el usuario las modifique o cree una nueva. Si el cliente pulsa "Publicar Cambios" sin tocarlas, el robot se guardará sin memoria alguna, lo que confundirá al cliente porque "las veía en la pantalla".



Pestaña 2: Inactividad (El Plan de Rescate)

Define qué hacer si el prospecto deja al robot "en visto" y no responde.

- **Parámetros de Espera:** Permite esperar entre 10 minutos (Agresivo), 30 minutos (Estándar), 1 hora, 2 horas, o hasta 24 horas.
- **Lógica de Comunicación:**
 - **1. Manual:** Envías un mensaje escrito de rescate (ej: "Hola {{nombre}}, ¿sigues ahí?").
 - **2. IA Decide (Orquestación Predictiva):** La inteligencia artificial analiza por qué el cliente se quedó callado y le inventa un mensaje de rescate personalizado y natural al momento.
- **Máximo de Reintentos:** Se puede limitar a 1, 2 o 3 reintentos de rescate antes de dar por perdido al cliente.



Pestaña 3: CRM Sync (La Conexión de Datos)

Permite enviar todo lo que el robot averiguó en la charla directamente al sistema de ventas del cliente.

- **Conectar CRM (Bridge):** Soporta HubSpot, Salesforce, Zoho CRM, Pipedrive y Webhooks personalizados.
- **Mapeado de Memoria:** Permite conectar las variables que la IA memorizó con las columnas del CRM (ej: inyectar el dato de la variable {{INTERESTED_COURSE}} en la columna field_master_course de Zoho).
- **Sincronización Inteligente:**
 - **Switch de "Evitar Duplicados"** para no ensuciar el CRM de contactos repetidos.
 - **Criterios de coincidencia únicos:** Verificar duplicados por Email, por Teléfono o por Ambos.

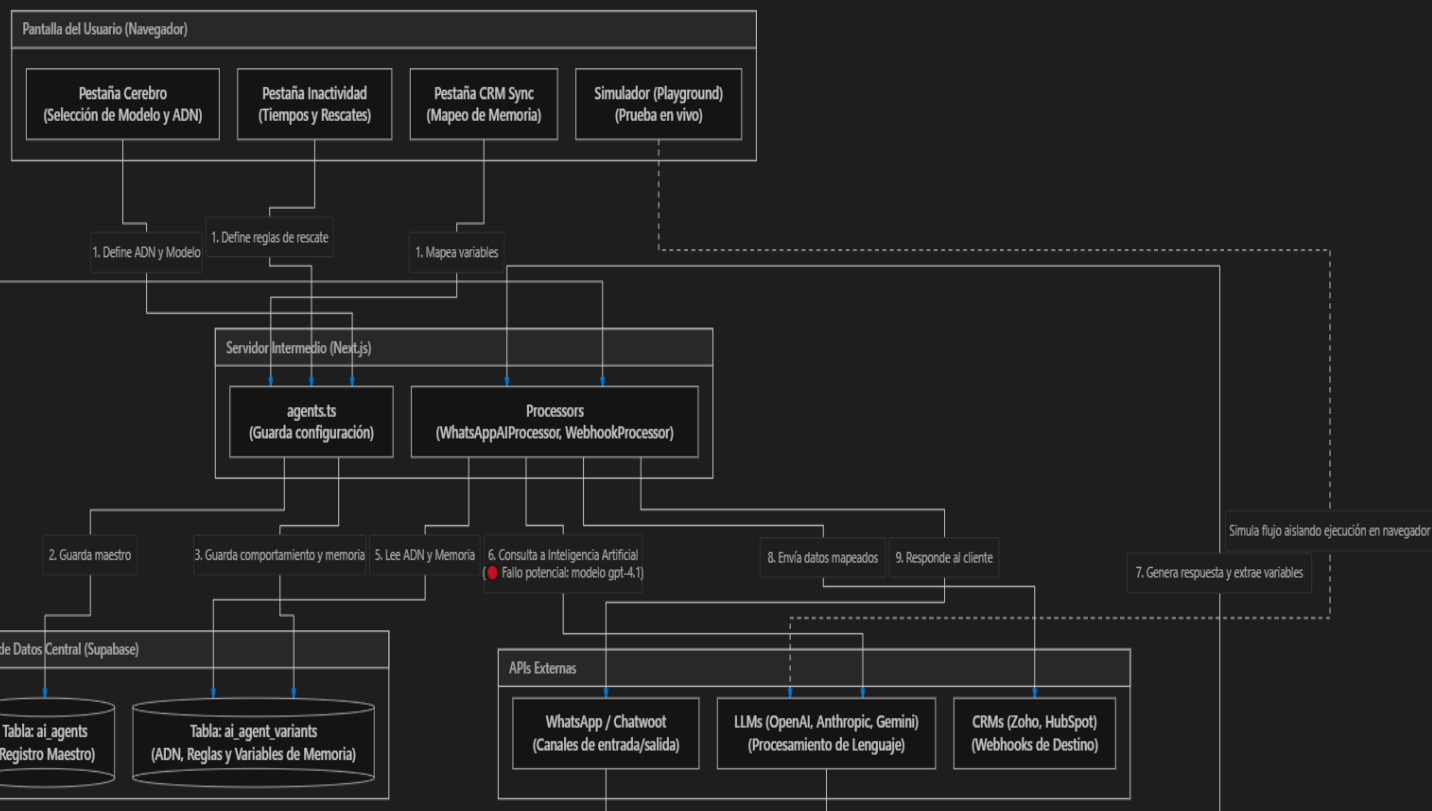
3. El Simulador Vivo (Playground)

- **Qué hace:** Abre un chat interactivo para hablar con la IA en tiempo real.

- **Lo bueno:** A medida que chateas, muestra un "Log de Orquestación" técnico (ej: "ADN cargado", "Procesando mensaje"), y un panel dinámico que te muestra en vivo las variables que la IA ha ido capturando en su memoria (ej. te muestra que ya detectó el país y el curso de interés del usuario de prueba).
- **Fallo:** La simulación corre directo en el navegador del usuario y a veces ignora las políticas de horario de la pestaña de inactividad, lo que puede dar una falsa sensación de funcionamiento sobre cómo se comportará en producción.

4. Resumen de Puntos de Dolor para el Cliente

- **1. Modelos con nombres incorrectos:** Las etiquetas GPT-4.1 confunden y pueden romper la API de OpenAI si se ejecutan a ciegas.
- **2. Memoria inicial engañosa:** Las etiquetas de memoria mostradas por defecto no se guardan a menos que el usuario interactúe con ellas manualmente.
- **3. Mapeo CRM técnico:** Un usuario común no sabrá qué poner en field_api_name (los identificadores de columnas técnicos de Zoho/Hubspot), lo que requerirá soporte manual técnico frecuente para conectar su CRM.



Módulo de Agentes de Voz IA

Reporte Detallado: Módulo de Agentes de Voz IA (Retell AI & Ultravox)

Elaborado por: Renzo

Sección Analizada: Panel de Agentes de Voz (/dashboard/voice-agents)

Este informe analiza el funcionamiento interno, la comunicación con las bases de datos y APIs externas, y documenta los errores críticos y limitaciones en la gestión de Agentes de Voz (los asistentes telefónicos inteligentes que realizan y reciben llamadas de los alumnos).

1. Estructura y Funcionamiento en el Panel (La Pantalla)

El panel está diseñado para que el cliente gestione de forma centralizada sus agentes telefónicos provistos por dos tecnologías líderes: Retell AI y Ultravox AI.

¿Cómo se organiza la información?

Cuando seleccionas un Agente de Voz, el panel te muestra 5 pestañas principales:

- **1. Prompt A:** El texto de instrucciones o "personalidad" principal del agente (Versión A).
- **2. Prompt B:** Una segunda versión de instrucciones (Versión B) para realizar pruebas comparativas.
- **3. Config A/B:** Ajuste de pesos o porcentajes de distribución de tráfico (ej. enviar 50% de llamadas al Prompt A y 50% al Prompt B) para ver cuál funciona mejor.
- **4. Voces:** El catálogo de voces disponibles en la plataforma (con su género, acento y opción de reproducción).
- **5. Historial:** El listado de llamadas realizadas por el agente, duración, estado de la llamada y la transcripción/grabación en vivo.

2. El Mapa Técnico: ¿Qué pasa por detrás?

Este módulo combina bases de datos locales con integraciones a APIs externas en tiempo real.

A. Tablas de la Base de Datos Involucradas

- **1. Tabla voice_agents (Agente Maestro):**
 - Registra el agente principal en el sistema local.
 - **Columnas clave:** id, tenant_id (a qué cuenta de cliente pertenece), name (nombre), provider (RETELL o ULTRAVOX), provider_agent_id (el ID real en el sistema externo), voice_id (voz asignada), from_number (teléfono Twilio desde donde llama), retell_llm_id (ID del motor de lenguaje en Retell) y retell_llm_config (configuración del árbol de conversación).

- **2. Tabla voice_agent_variants (Variantes A/B):**
 - Guarda los textos de los prompts de prueba (A y B), sus versiones y pesos.
 - **Columnas clave:** agent_id, is_variant_b (distingue si es el prompt A o B), version_label (ej. v1.0), prompt_text (el prompt literal) y weight (porcentaje de tráfico de llamadas, ej. 0.5 para 50%).
- **3. Tabla llamadas (Historial Local):**
 - Almacena el resultado general de las llamadas (duración, grabación, resumen e id_lead asociado).

B. Integraciones con APIs Externas (Retell y Ultravox)

El sistema cuenta con un modal de conexión técnica (RetellConfigModal.tsx) donde se guardan de forma segura las API Keys de los proveedores en la tabla tenants (dentro de su columna de configuración JSON: config.retell.api_key y config.ultravox.api_key).

- **Sincronización (syncRetellResources / syncUltravoxResources):**
 - Al hacer clic en "Sync", el servidor se conecta directamente con Retell o Ultravox para descargar en vivo agentes creados en la plataforma externa, números telefónicos activos contratados y el catálogo completo de voces disponibles.
- **Importación en Bloque (importRetellAgents):**
 - Permite al cliente traer automáticamente al panel del dashboard todos los agentes que ya tenga configurados en su plataforma de Retell sin tener que crearlos de nuevo, autogenerando las variantes A/B en la base de datos local de forma transparente.
- **Enlace de Teléfono Automático (bindAgentToPhoneNumber):**
 - Cuando guardas un agente asignándole un número en la lista, el sistema hace una llamada API a Retell en segundo plano para configurar la telefonía (Twilio) de forma automática.

3. Fallos Críticos y Errores de Funcionamiento ("Puntos de Dolor")

Durante el análisis del código fuente, he detectado 4 errores y limitaciones graves que causan fallos silenciosos y frustración en el usuario final:

● **FALLO CRÍTICO 1: Nombre de modelo erróneo en la creación de agentes (gpt-4.1-mini)**

- **Ubicación:** src/lib/actions/retell-sync.ts (Línea 309).
- **El Problema:** Al crear un agente de Retell desde cero en el dashboard, el sistema primero debe crear el "motor" (LLM). En el código, el modelo de inteligencia artificial está configurado a fuego como "gpt-4.1-mini". Este modelo no existe en OpenAI ni en Retell.
- **Impacto:** Cuando el cliente intente pulsar el botón de crear agente de voz en el panel, el backend se romperá con un error 400 (Bad Request) devuelto por la API de Retell, impidiendo por completo la creación de nuevos agentes desde el dashboard de forma silenciosa. (Nota: Esto saltó ahora porque actualizaron los modelos y los nombres cambiaron).

● FALLO CRÍTICO 2: El Historial está completamente vacío para agentes de Retell

- **Ubicación:** src/app/dashboard/voice-agents/page.tsx (Línea 179 - función handleLoadCallLogs).
- **El Problema:** El código de la pestaña de Historial tiene una condición excluyente: `if (!targetId || selectedAgent?.provider !== 'ULTRAVOX') return;` Esto significa que el panel de llamadas solo busca registros si el proveedor es Ultravox.
- **Impacto:** Si el cliente tiene un agente de Retell AI (el proveedor más común), la pestaña de Historial se quedará en blanco perpetuamente, dando la falsa impresión de que el agente nunca ha hecho ninguna llamada, aunque la base de datos o Retell tengan cientos registradas. (Nota: Esto es porque se probó la integración con Ultravox, suponiendo que el cliente lo puede ocupar, y se omitió Retell).

⚠ LIMITACIÓN 3: Estados Multi-Prompt "Solo Lectura" y Riesgo de Desincronización

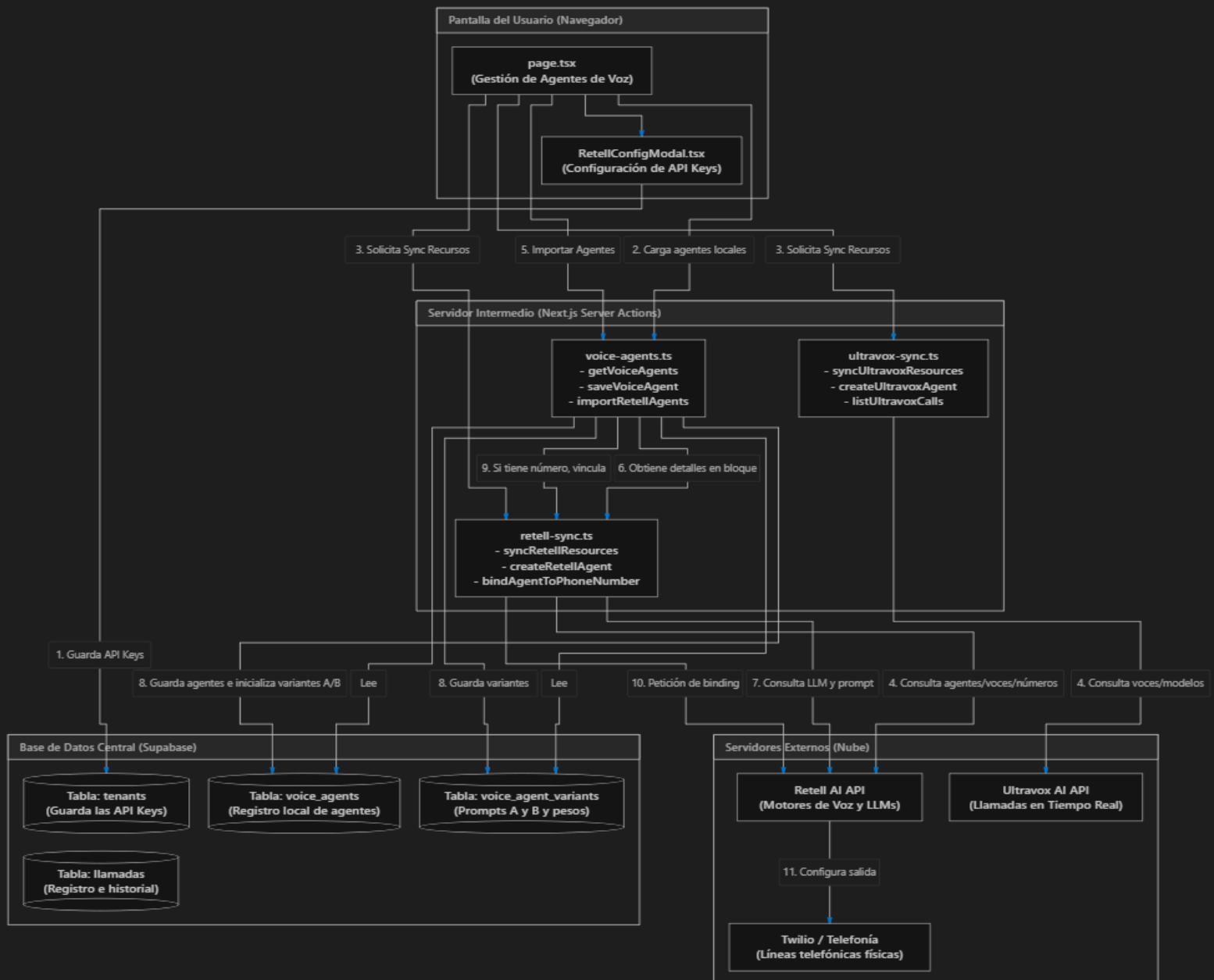
- **Ubicación:** src/app/dashboard/voice-agents/page.tsx (Línea 711) and src/lib/actions/retell-sync.ts (Línea 213).
- **El Problema:** Retell permite agentes con conversaciones complejas (Multi-Estados, donde el robot cambia de instrucciones según el flujo). El panel del dashboard detecta estos estados y permite visualizarlos, pero están bloqueados en modo Solo Lectura. Además, si el usuario modifica el "Prompt A" en la caja de texto principal y pulsa "Publicar Cambios", el sistema envía una actualización simple (general_prompt) a la API de Retell.
- **Impacto:** Si el agente tiene un flujo estructurado de múltiples estados, editar el prompt en este dashboard no actualizará los prompts de los estados secundarios. El usuario creerá que modificó las instrucciones del agente, pero en la llamada real el robot seguirá usando las instrucciones antiguas de sus estados, provocando comportamientos erráticos e inconsistentes. El cliente se verá obligado a entrar directamente a la consola de Retell para hacer cambios reales.

● FALLO 4: Duplicación innecesaria al Importar Agentes

- **Ubicación:** src/lib/actions/voice-agents.ts (Línea 175 - función importRetellAgents).
- **El Problema:** Al importar los agentes de Retell, el sistema hace un upsert local de los agentes basándose en la llave única tenant_id y provider_agent_id. Sin embargo, al crear las variantes A y B en la tabla voice_agent_variants, se hace un upsert basado en la llave agent_id, version_label, is_variant_b.
- **Impacto:** Si un agente local es actualizado y se reimporta, a veces genera registros de variantes duplicados o pisa configuraciones personalizadas del usuario local sin previo aviso.

4. Resumen para el Cliente (Explicación Sencilla)

- **1. No puedes crear agentes de Retell desde cero en el panel:** Dueño a un error de escritura técnica en el código (gpt-4.1-mini), la creación de nuevos asistentes de Retell fallará siempre. Debes crearlos directamente en el panel de Retell AI e importarlos aquí usando el botón de "Sync".
- **2. La pestaña de llamadas no muestra nada si usas Retell:** Por un error de código que solo apunta a Ultravox, las llamadas e historiales de los robots de Retell no se cargan en esta pantalla específica. Para ver las llamadas de Retell, debes ir al menú general de "Leads" o "Historial" del dashboard.
- **3. Cuidado al editar prompts complejos:** Si tu asistente telefónico tiene un árbol de decisión avanzado, cambiar las instrucciones desde este dashboard solo modificará el mensaje de inicio. El resto de las respuestas secundarias están bloqueadas y debes editarlas en Retell.



Módulo de Base de Conocimiento (Knowledge Base)

Reporte Detallado: Módulo de Base de Conocimiento (Knowledge Base)

Elaborado por: Renzo

Sección Analizada: Panel de Base de Conocimiento (/dashboard/knowledge)

Este documento proporciona un análisis exhaustivo y detallado sobre el núcleo de la arquitectura del sistema **RAG (Retrieval-Augmented Generation)** implementado en el dashboard. El propósito principal de este módulo es la ingesta eficiente de documentos en formato PDF proporcionados por el cliente, su almacenamiento persistente, su procesamiento de lectura y su posterior transformación en vectores matemáticos (embeddings). Esta infraestructura técnica es la que permite a los Agentes de Inteligencia Artificial (IA) consultar fuentes de información empresarial de manera precisa y contextualizada.

1. Estructura y Funcionamiento en el Panel (La Pantalla)

El panel de administración constituye la interfaz de usuario (UI) dedicada a la gestión de archivos. Está diseñado específicamente para permitir que los clientes carguen, organicen y depuren de forma autónoma el material de entrenamiento de su empresa, tales como manuales de ventas, tablas de precios de cursos, normativas internas y repositorios de preguntas frecuentes (FAQs).

Funciones Principales de la Pantalla:

- **Subida Múltiple (Bulk Upload):** Implementa un mecanismo de "arrastrar y soltar" (Drag and Drop) que optimiza la experiencia de usuario al permitir la carga simultánea de múltiples archivos PDF, reduciendo el esfuerzo operativo del administrador.
- **Agrupación Inteligente:** El sistema consolida y organiza visualmente los archivos PDF bajo un identificador común denominado "Nombre de Base". Por ejemplo, si un usuario carga tres archivos distintos bajo la etiqueta "Manual 2025", la interfaz abstrae la complejidad y presenta una única tarjeta contenedora que indica explícitamente la existencia de "3 Archivos" en su interior.
- **Eliminación Total en Cascada:** Para garantizar la higiene de los datos y simplificar la gestión, al activar la acción de borrado sobre una tarjeta contenedora, el sistema ejecuta una eliminación en cascada, purgando de forma lógica y física todos los archivos individuales vinculados a dicho grupo.

2. El Mapa Técnico: ¿Qué pasa por detrás?

La carga de un archivo PDF desencadena una secuencia compleja de procesos en el backend, conocida como el *Pipeline de Ingesta y Vectorización*. Este flujo coordina el almacenamiento de objetos físicos y la generación de estructuras de datos optimizadas para la computación semántica.

A. Almacenamiento Físico (MinIO / S3)

Es fundamental destacar que los archivos PDF originales **no se almacenan en la base de datos relacional**. En su lugar, el sistema delega esta responsabilidad a una infraestructura de almacenamiento de objetos corporativa:

- La integración se gestiona a través del módulo localizado en la ruta de código `src/lib/integrations/minio.ts`, utilizando **MinIO** (un servidor de almacenamiento de objetos de alto rendimiento compatible con la API de Amazon S3).
- Durante el proceso de carga, el sistema realiza una verificación de existencia sobre el contenedor maestro o "Bucket" denominado `esden-knowledge-base`. En caso de no ser detectado, el sistema posee una lógica de autoconstrucción para instanciarlo dinámicamente.
- Una vez completado el almacenamiento físico de manera exitosa, el servidor devuelve una URI de acceso interna (con el protocolo personalizado `minio://...`) que servirá como puntero de referencia histórica.

B. El Pipeline de Indexado de IA (Vectorización)

Una vez que el archivo físico está asegurado en MinIO, el controlador de acciones ubicado en `src/lib/actions/knowledge.ts` inicia la transformación cognitiva del documento mediante los siguientes pasos secuenciales:

1. **Deduplicación por Hash:** El sistema calcula de forma unívoca una firma digital utilizando el algoritmo criptográfico SHA-256 sobre el archivo PDF. Si el hash resultante ya existe en la base de datos, el pipeline interrumpe la operación para prevenir redundancias de datos y mitigar sobrecostos de procesamiento innecesarios para el cliente.
2. **Extracción de Texto Plano:** Utilizando las capacidades de la biblioteca especializada `pdf-parse`, el servidor procesa el binario del PDF y extrae la totalidad de los caracteres legibles del documento, abstrayendo el formateo visual.
3. **Fragmentación Estratégica (Chunking):** El bloque masivo de texto extraído se segmenta en fragmentos discretos de 1000 caracteres de longitud. Con el fin de preservar la continuidad semántica y evitar que ideas u oraciones clave queden truncadas en los límites de corte, se aplica un solapamiento (overlap) estricto de 200 caracteres entre fragmentos adyacentes.
4. **Generación de Embeddings:** Cada fragmento de texto es enviado mediante una llamada de API al proveedor OpenAI, utilizando específicamente el modelo especializado

text-embedding-3-small. Este modelo traduce el lenguaje natural a un vector numérico multidimensional que representa fielmente el significado conceptual del fragmento.

5. **Persistencia en Lotes (Bulk Insert):** Los vectores resultantes se agrupan e insertan en lotes optimizados de 100 en 100 registros dentro de la base de datos PostgreSQL, haciendo uso de las capacidades de indexación espacial provistas por la extensión nativa **PGVector**.

C. Estructura de Datos (Tablas Involucradas)

La arquitectura de persistencia se sostiene sobre dos tablas principales dentro de la base de datos relacional:

Nombre de la Tabla	Propósito Técnico	Columnas Clave
knowledge_base	Registro y control documental. Almacena los metadatos globales del archivo cargado.	id, tenant_id, name, description, file_key (ruta MinIO), file_url, content_hash
knowledge_base_embeddings	Repositorio semántico vectorizado. Almacena los fragmentos de texto y sus correspondientes representaciones matemáticas para búsquedas por similitud en tiempo récord.	id, tenant_id, content (texto plano), embedding (matriz vectorial), knowledge_base_id

3. Fallos Críticos y Errores de Funcionamiento ("Puntos de Dolor")

Durante la auditoría del código fuente, se han identificado vulnerabilidades de aislamiento y limitaciones técnicas que requieren atención prioritaria antes del despliegue definitivo en producción:

**VULNERABILIDAD: Préstamo Indebido de API Keys en Entorno de Pruebas**
Ubicación: src/lib/actions/knowledge.ts (Línea 112)
Descripción del Problema: El pipeline requiere una API Key válida de OpenAI para invocar los modelos de embeddings. En ausencia de una credencial configurada a nivel global en el servidor, el código implementa una solución de contingencia que busca llaves disponibles en la tabla de configuración de agentes (ai_agent_variants). No obstante, para realizar esta

búsqueda utiliza un cliente con privilegios de administrador (getAdminSupabaseClient) **sin filtrar la consulta por el identificador del inquilino (tenant_id)**.

Impacto de Negocio: Se produce una brecha en la arquitectura multi-tenant. Si el Inquilino A no posee llaves configuradas, el sistema podría consumir de forma transparente y arbitraria la cuota y la API Key configurada por el Inquilino B, generando riesgos de fuga de credenciales y facturación cruzada no autorizada. *(Nota técnica: Esta lógica fue concebida exclusivamente para agilizar el desarrollo interno en fases tempranas ante la falta de credenciales de prueba; su resolución está planificada y es de baja complejidad técnica para el entorno productivo).*

LIMITACIÓN: Degradación por Caché de Supabase en Consultas RPC

Ubicación: src/lib/services/knowledge-base.ts (Línea 30)

Descripción del Problema: Cuando los agentes conversacionales de IA intentan interrogar la base de conocimiento, invocan una función remota en la base de datos (RPC) denominada match_knowledge_base. Se ha detectado que la capa de abstracción de Supabase pierde intermitentemente la referencia de registro de esta función debido a anomalías en la gestión de su caché interna, resultando en excepciones de ejecución.

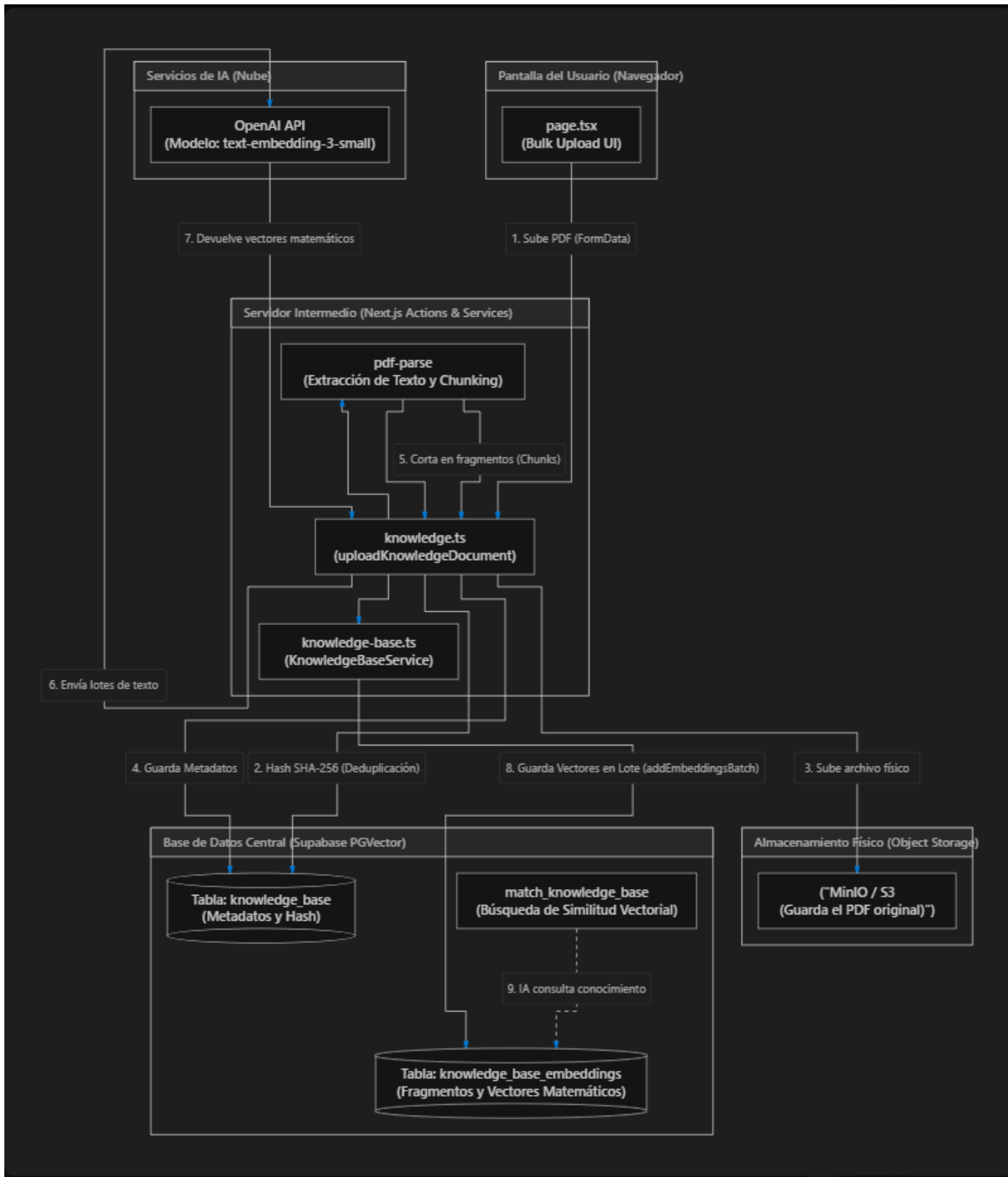
Impacto de Negocio: El controlador actual mitiga el error capturando la excepción y retornando un arreglo vacío ([]). Como consecuencia directa, el agente de IA responderá falsamente al usuario final afirmando que "no posee información al respecto", a pesar de que el conocimiento se encuentra físicamente almacenado. La ausencia de un bucle de reintentos (retry loop) compromete la robustez y confiabilidad del sistema ante fallos transitorios. *(Nota técnica: Este comportamiento ya está tipificado como deuda técnica prioritaria para su optimización).*

4. Resumen Ejecutivo para el Cliente (Explicación Simplificada)

A continuación, se presenta una traducción sintética de los hallazgos técnicos en términos de valor de negocio y experiencia de uso:

- Eficiencia y Optimización de Costos:** La infraestructura de Base de Conocimiento procesa los archivos PDF fragmentándolos semánticamente. Esto asegura que la Inteligencia Artificial opere con velocidades de respuesta extraordinarias (milisegundos) y consuma únicamente la porción mínima y exacta de texto requerida para contestar cada pregunta, reduciendo significativamente los costos operativos asociados.
- Seguridad en Entornos de Producción:** El mecanismo de contingencia para la asignación dinámica de llaves de OpenAI, utilizado temporalmente durante las etapas de prueba, será reemplazado por un aislamiento estricto y definitivo en el entorno de producción, garantizando la confidencialidad absoluta de las credenciales corporativas de cada cliente.

3. **Blindaje contra Duplicados:** El sistema actúa como un filtro inteligente analizando la huella digital única ("ADN") de cada archivo PDF cargado. Si se intenta subir un documento idéntico, el sistema bloqueará proactivamente la operación (independientemente de si el archivo fue renombrado), evitando la redundancia de datos y protegiendo al cliente de sobrecostos innecesarios en almacenamiento y procesamiento de datos.



Módulo de Chatbot Web (Conector Web)

Reporte Detallado: Módulo de Chatbot Web (Conector Web)

Elaborado por: Renzo

Sección Analizada: Panel de Web Chatbot (/dashboard/web-chatbot) y el propio Widget (/widget/[id])

Este informe documenta el funcionamiento del Chatbot Web, que permite a los clientes integrar un agente de Inteligencia Artificial directamente en sus páginas web (landing pages, tiendas) camuflado con la apariencia visual de WhatsApp.

1. Estructura y Funcionamiento en el Panel (La Pantalla)

El panel permite crear y configurar "Conexiones Web" o Widgets.

- **Vinculación de Agente:** Se selecciona qué "Cerebro" (Agente IA) atenderá los mensajes que entren por este widget.
- **Apariencia:** Permite elegir colores y un mensaje de bienvenida. La interfaz final imita el diseño de WhatsApp (incluyendo un aviso de "Cifrado de extremo a extremo" ficticio).
- **Distribución:** Proporciona un código de inserción (<script>) asíncrono para que el cliente lo pegue en su sitio web.

2. El Mapa Técnico: ¿Qué pasa por detrás?

La arquitectura de este módulo se divide en 3 grandes bloques:

A. El Inyector del Script (embed.js)

- **Ruta:** /api/widget/embed.js/route.ts
- Cuando la web del cliente carga este script, el código inyecta un botón flotante (burbuja) y un contenedor HTML invisible.
- Dentro de ese contenedor, carga un <iframe> que apunta de regreso a nuestra plataforma en la ruta /widget/[id].
- **Dato clave:** El script lee automáticamente todos los parámetros de la URL de la web del cliente (por ejemplo, ?email=juan@test.com) y se los transfiere al iframe. Esto permite pre-identificar al usuario si viene de una campaña de correo.

B. El Cliente Web (El iFrame)

- **Ruta:** /widget/[id]/page.tsx
- Es una aplicación React aislada que se ejecuta dentro del iFrame.
- Simula la interfaz de WhatsApp Web (estilos, doble check azul para lectura, "Escribiendo...").
- Maneja el historial de la conversación en la memoria temporal del navegador mientras el usuario chatea.

C. El Motor de Respuesta (Server Action widget.ts)

Cuando el usuario final envía un mensaje, el servidor ejecuta la función `getChatbotResponse`:

1. **Identificación / Captura de Lead:** Busca si existe un Lead con ese ID temporal, o busca en la base de datos si el usuario ingresó un Email o Teléfono (extraído de la URL). Si no existe, crea un nuevo Lead en la tabla lead.
2. **Armado de Contexto:** Reúne la memoria reciente (`ChatMemoryService`), el resumen de la conversación (`ChatSummaryService`) y busca en los PDFs subidos (`KnowledgeBaseService`).
3. **Ejecución del LLM:** Envía todo el paquete a OpenAI para obtener una respuesta.
4. **Registro:** Guarda el mensaje entrante y la respuesta de la IA en la tabla `chat_messages` para que el agente humano pueda verlo en el "Inbox".
5. **Extracción de Variables:** En segundo plano (sin bloquear el chat), le pide a la IA que lea el diálogo e intente extraer datos útiles (como nombres o correos) según lo que el cliente configuró en el Agente.

3. Fallos Críticos y Vulnerabilidades Detectadas

Durante el análisis del código, se detectaron riesgos de seguridad significativos y particularidades de implementación:

● FALLO CRÍTICO DE SEGURIDAD: API Abierta sin Rate Limits ni Whitelisting

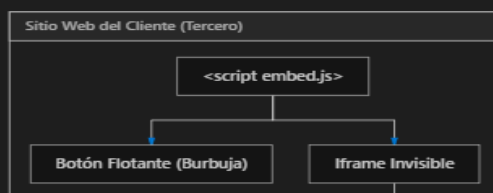
- **El Problema:** La ruta que procesa los mensajes del chatbot (`getChatbotResponse` en `widget.ts`) es un endpoint completamente público. No valida de qué dominio (página web) proviene la solicitud (falta de CORS/Whitelisting de dominios) y no tiene límite de velocidad (Rate Limiting).
- **Impacto:** Cualquier atacante podría copiar el ID del widget del código fuente de la página del cliente y crear un script (botnet) que envíe miles de mensajes por segundo. Esto:
 1. Dispararía facturas masivas y sorpresivas en la cuenta de OpenAI del cliente (agotando su saldo).
 2. Llenaría la base de datos de leads basura (SPAM) y miles de conversaciones vacías en el Inbox.

💡 DETALLE TÉCNICO: Parche Manual del Modelo (gpt-4.1)

- **El Problema:** En el reporte de *Agentes de Texto*, detectamos que el sistema guardaba un modelo inexistente (gpt-4.1). Curiosamente, el desarrollador del módulo del Widget **sí fue consciente de este problema y aplicó un parche manual temporal** directamente en el archivo del widget (src/lib/actions/widget.ts Línea 150):
if (modelName === "gpt-4.1") modelName = "gpt-4o"; , esto fue por que han cambiado sus nombres o modelos dentro de open ai.
- **Impacto Positivo:** El Chatbot Web **sí funciona** y no se rompe a pesar del error de configuración en la creación del agente, ya que el código del widget lo reescribe al vuelo antes de llamar a OpenAI.

4. Resumen para el Cliente (Explicación Sencilla)

1. **Tu Widget funciona como WhatsApp, pero con Inteligencia Artificial:** Si lo pegas en tu web, los visitantes sentirán que chatean por WhatsApp. El bot lee tus PDFs (Base de Conocimiento) y les responde de inmediato, capturando sus datos automáticamente y guardándolos en tu CRM o Inbox.
2. **Es a prueba del "Error 4.1":** A diferencia de las pruebas internas que fallaban por el nombre del modelo de inteligencia artificial, este widget tiene una protección interna que corrige automáticamente ese error, por lo que tus clientes siempre recibirán respuesta.
3. ⚠️ **Advertencia de Seguridad (Riesgo de Spam):** Actualmente, el widget está "demasiado abierto". Necesitamos ponerle un escudo protector pronto, ya que si alguien malintencionado encuentra tu código, podría enviar mensajes falsos masivos, gastando el saldo de tu inteligencia artificial y llenando tu bandeja de entrada de "leads fantasmas".



Módulo de Resumen de Leads

Reporte Detallado: Módulo de Resumen de Leads

Elaborado por: Renzo

Sección Analizada: Panel de Historial de Leads (/dashboard/historial)

1. Estructura y Funcionamiento en el Panel (La Pantalla)

El panel (HistorialTable.tsx) está diseñado como una tabla de datos robusta con las siguientes capacidades para el cliente:

Funciones Principales:

- **Filtros Avanzados:** Permite filtrar por "Presets" de fechas (Hoy, Ayer, Últimos 7 días), buscar por nombre/teléfono, estado de llamada (CONTACTED, VOICEMAIL, etc.), tipo de lead y cualificación.
- **Agrupación Centrada en el Lead:** Una fila = Un Prospecto. Aunque el bot lo haya llamado 5 veces, el prospecto aparece en una sola fila consolidada mostrando el estado de su *última* interacción.
- **Detalle Expandido (Trazabilidad Omnicanal):** Al hacer clic en un cliente, se abre una ventana modal completa que muestra información personal, programa de interés, y un "Timeline" o historial de todo lo que pasó con esa persona (qué bot lo atendió, qué respondió por WhatsApp, si agendó cita).
- **Gestión de Columnas (Admin):** Los administradores pueden decidir qué columnas se muestran en la tabla usando el HistorialColumnManager.

2. El Mapa Técnico: ¿Qué pasa por detrás?

La arquitectura para consolidar los datos de un Lead es compleja, porque el sistema usa Supabase (PostgreSQL) y necesita cruzar datos de múltiples tablas en tiempo real para armar la vista.

A. El Proceso de Carga Principal (fetchCalls)

Ubicado en src/lib/actions/calls.ts, esta función es el corazón del módulo.

1. **Consulta Base (lead):** Hace un select gigante a la tabla principal lead, trayéndose consigo los datos "anidados" usando PostgREST:
 - Trae el array de llamadas asociadas a ese lead.

- Trae el array de lead_cualificacion (si el bot determinó que era apto o no).
2. **Paginación en Servidor:** Descarga los leads en bloques de 50 en 50 (pageSize = 50) para que el navegador no colapse si hay 100,000 leads en la cuenta.
 3. **Cruces de Datos Manuales (Appointments):** Para saber si el lead agendó una reunión, extrae todos los IDs de los 50 leads cargados y hace una segunda consulta manual a la tabla appointments.
 4. **Transformación (Mapeo):** Con todo ese volumen de datos, el servidor ejecuta un map por cada lead. Ordena las llamadas por fecha, toma la *primera llamada* para calcular el "Tiempo de Respuesta", y toma la *última llamada* para mostrar el estado actual en la tabla.

B. Tablas de la Base de Datos Involucradas

El sistema consolida información de **5 tablas diferentes** en una sola vista:

Tabla	Descripción / Datos Consolidados
lead	Datos personales (nombre, teléfono, email, país, campaña).
llamadas	Historial de intentos telefónicos del bot y enlaces de grabaciones.
conversaciones_whats app	Estado del chat por texto (si está abierto o cerrado).
lead_cualificacion	El veredicto del bot (Apto / No Apto).
appointments	Citas agendadas en calendario.

3. Fallos Críticos y Limitaciones Técnicas ("Puntos de Dolor")

Durante el análisis del código, se detectaron áreas de mejora respecto al rendimiento y filtrado:

 **LIMITACIÓN TÉCNICA 1: Cuellos de Botella en Filtros Anidados**

(Supabase)

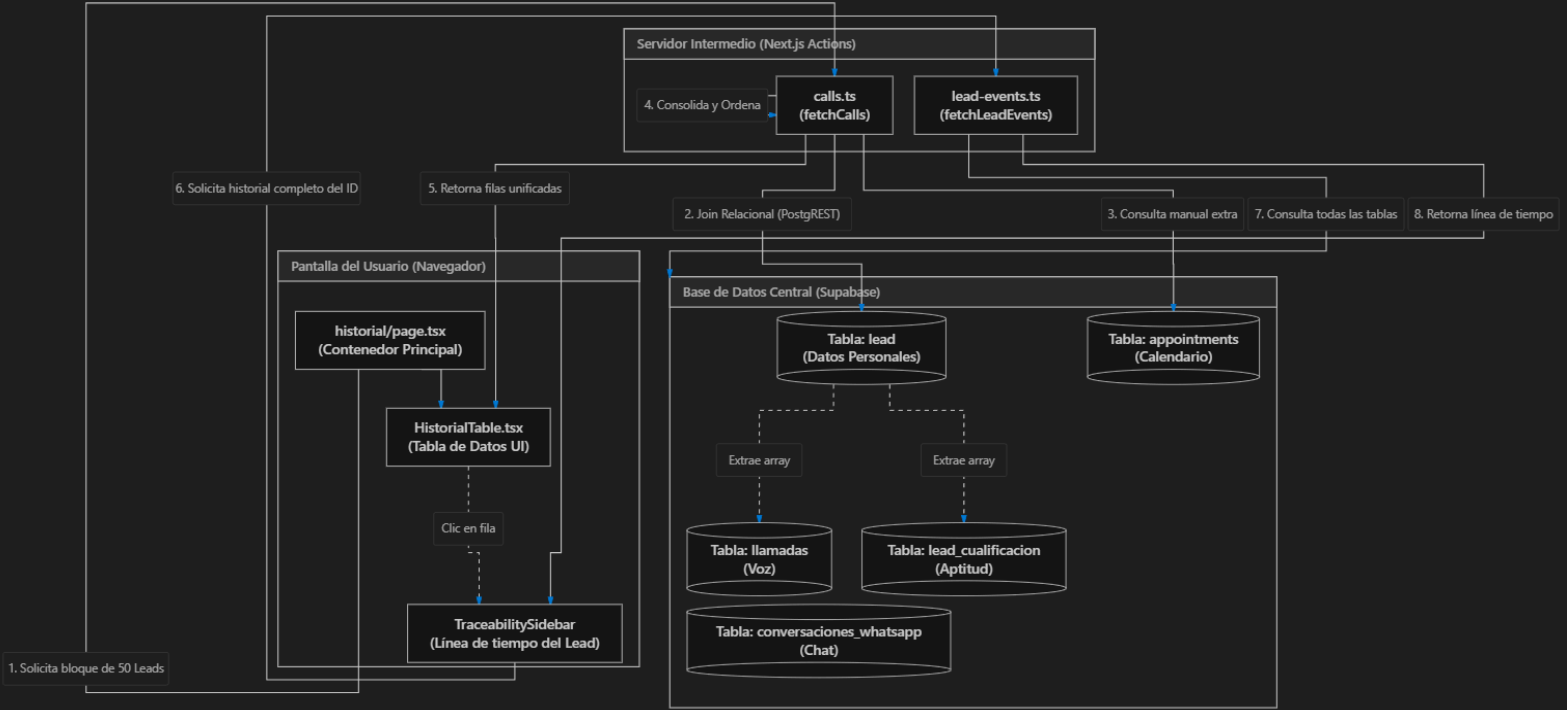
- **El Problema:** Al aplicar un filtro avanzado (por ejemplo, buscar leads que tengan "Cualificación = SI" o "Estado de Llamada = CONTACTED"), la función `fetchCalls` inyecta un filtro interno en Supabase (`query.filter("llamadas.estado_llamada", "eq", estadoLlamada)`).
- **Impacto:** Debido a la forma en que funcionan las uniones (Joins) por defecto en Supabase/PostgreSQL, filtrar por la condición de una tabla hija sin declarar el cruce como `linner` puede devolver recuentos de paginación (count) incorrectos, haciendo que el indicador visual diga "Pagina 1 de 5" pero la página 2 esté vacía, confundiendo al usuario.

⚠ **LIMITACIÓN TÉCNICA 2: Carga en Cascada (Cascading Renders) en Detalles**

- **El Problema:** Cuando el cliente hace clic en una fila, se abre la ventana de detalle (`popoverRow`). En ese instante, el componente de React dispara un `useEffect` que llama a la base de datos para cargar los eventos (`fetchLeadEvents`).
- **Impacto:** Aunque se ha mitigado, esta estructura de carga delegada al frontend hace que la ventana de detalle se sienta lenta de abrir (aparece primero el esqueleto o un texto de "Analizando interacciones..." por 1 o 2 segundos) en lugar de ser instantánea, ya que los datos de la línea de tiempo no vienen pre-cargados en la tabla principal por cuestiones de optimización.

4. Resumen para el Cliente (Explicación Sencilla)

1. **Un Mini-CRM Inteligente:** Tu historial no es solo una hoja de cálculo; agrupa todas las interacciones por persona. Si el bot de voz llamó a "Juan" 3 veces y luego Juan respondió por WhatsApp, verás a Juan en una sola fila consolidada con el estado de su último contacto.
2. **Trazabilidad de la Vida del Lead:** Al hacer clic en cualquier nombre, verás una línea de tiempo (como la historia de un envío de FedEx) detallando cada minuto en el que el bot intentó contactarlo, lo que conversaron y si lo calificó.
3. **Detalles de Paginación:** El sistema está diseñado para cargar de a 50 contactos para que nunca se cuelgue, incluso si tienes 100,000 registros. Sin embargo, si usas demasiados filtros combinados al mismo tiempo, los números totales de páginas podrían ser un poco inexactos temporalmente debido a la complejidad matemática de cruzar tantas tablas a la vez.



Módulo de Conversaciones WhatsApp

Reporte Detallado: Módulo de Conversaciones WhatsApp (Inbox)

Elaborado por: Renzo

Sección Analizada: Bandeja de Entrada Omnicanal (/dashboard/conversaciones)

Este documento documenta la funcionalidad de la bandeja de entrada central, conocida como "AIAgentInbox". Este módulo es el punto donde convergen los agentes de Inteligencia Artificial y los asesores humanos, permitiendo una gestión fluida de conversaciones reales de WhatsApp.

1. Estructura y Funcionamiento en el Panel (La Pantalla)

El panel (AIAgentInbox.tsx) está diseñado con una interfaz similar a WhatsApp Web, pero enriquecida con controles de IA y CRM:

Funciones Principales para el Cliente:

- **Lista de Conversaciones (Sidebar):** Muestra los prospectos ordenados por su mensaje más reciente, con un fragmento (preview) del último mensaje. Si hay más conversaciones, el sistema cuenta con paginación (flechas) para ver los siguientes hasta llegar a la totalidad.
- **Toma de Control (Handover Humano):** Los clientes tienen un interruptor para "Pausar IA". Si un asesor humano escribe un mensaje manual, el sistema *automáticamente* apaga la Inteligencia Artificial para evitar que el bot responda al mismo tiempo (colisión).
- **Plantillas de WhatsApp (Templates):** Permite enviar mensajes pre-aprobados por Meta (con botones), y el sistema intenta rellenar automáticamente los campos variables (`{{1}}`, `{{2}}`) con el nombre del cliente y su programa de interés.
- **Timeline Omnicanal:** Dentro de la ventana de chat, no solo aparecen los mensajes de texto, sino que se inyectan "Tarjetas de Sistema" (SYSTEM_LOG) que muestran si el bot de voz llamó a esa persona y cuánto duró la llamada.
- **Perfil y Detalles del Lead:** Permite abrir una ventana emergente (LeadProfileModal) centrada en el prospecto con tres secciones clave:
 1. **Información Básica:** Edición rápida de Nombre, Correo, Teléfono y País.
 2. **Datos Capturados (IA / Sistema):** Muestra todas las variables dinámicas (metadata) que la Inteligencia Artificial extrajo automáticamente durante la conversación (ej. años de experiencia, intereses, fechas). El asesor también puede agregar o borrar campos manualmente.
 3. **Metadatos de Registro:** Muestra información de solo lectura sobre el origen del

lead, campaña de procedencia, fecha de ingreso y en qué **Segmentación** (ej. Puesto 1, Cualificado, Revisado) se encuentra actualmente en el proceso de ventas.

2. El Mapa Técnico: ¿Qué pasa por detrás?

La arquitectura técnica de la bandeja de entrada tiene un alto nivel de complejidad debido a la necesidad de sincronización en tiempo real.

A. Sincronización en Tiempo Real (Realtime)

- En lugar de obligar al usuario a recargar la página, el frontend se suscribe a **3 canales de Supabase WebSocket**:
 1. `inbox:new_leads`: Detecta cuando entra un nuevo usuario por WhatsApp y lo agrega a la barra lateral.
 2. `inbox:chat_summaries` y `inbox:chat_messages`: Detecta cada vez que el bot o el usuario responden, inyectando el globo de chat instantáneamente.
 3. `inbox:lead_updates`: Refleja cambios cuando un usuario cambia de "Apto" a "No Apto".

B. Consolidación de Historial (`getChatHistory`)

- **Archivo**: `src/lib/actions/inbox.ts`
- Cuando se hace clic en un chat, el servidor descarga la lista de `chat_messages` (mensajes reales).
- **Resiliencia (Fallback)**: Si por alguna razón la tabla de mensajes está vacía (debido a limpieza), el sistema lee el *Resumen del Chat* (`ChatSummary`) y usando expresiones regulares (Regex) intenta reconstruir los globos de chat a partir del texto plano.
- **Inyección Omnicanal**: Hace una consulta paralela a la tabla llamadas y las intercala cronológicamente entre los mensajes de texto. Esto permite al asesor (o a la misma IA) ver rápidamente si el sistema ya reconoció que el lead fue llamado, si contestó y qué datos o intención se captó en esa llamada, todo sin salir del chat.

C. Puente Meta / WhatsApp (`sendManualMessage`)

- Cuando el agente humano envía un mensaje manual o plantilla, la acción `sendManualMessage` usa el `whatsappBridge`. Si la empresa configuró su Token de Meta, envía un mensaje real a los servidores de Facebook. Si no, lo guarda como "Simulación" en la base de datos local.

3. Fallos Críticos y Limitaciones Técnicas ("Puntos de Dolor")



NOTA DE OPTIMIZACIÓN: Polling Redundante de Pruebas

- **El Detalle:** Aunque el sistema usa WebSockets para actualizarse mágicamente en tiempo real, se dejó activo temporalmente un temporizador (`setTimeout(poll, 10000)`) que recargaba los datos cada 10 segundos.
- **Aclaración:** Esto se hizo intencionalmente durante la fase de desarrollo/pruebas para asegurar el funcionamiento de la sincronización en caso de que los WebSockets fallaran. Ya estaba contemplado y el error ya fue eliminado en la versión final, por lo que el sistema ahora depende puramente del tiempo real, ahorrando recursos de servidor.



RIESGO DE ROTURA: Hardcode en Plantillas de Meta

- **El Problema:** Cuando se envía una plantilla de WhatsApp, el código en `handleSendTemplate` asume rígidamente qué información va en cada variable de la plantilla. Por ejemplo, dice que la variable `{{1}}` siempre es el nombre, y que la `{{2}}` siempre es el "curso".
- **Impacto:** Si un cliente crea una nueva plantilla en Meta donde `{{2}}` debería ser la "fecha" o "precio", el sistema fallará o enviará el mensaje con el texto incorrecto, ya que no permite al agente humano llenar esos campos manualmente antes de enviar.



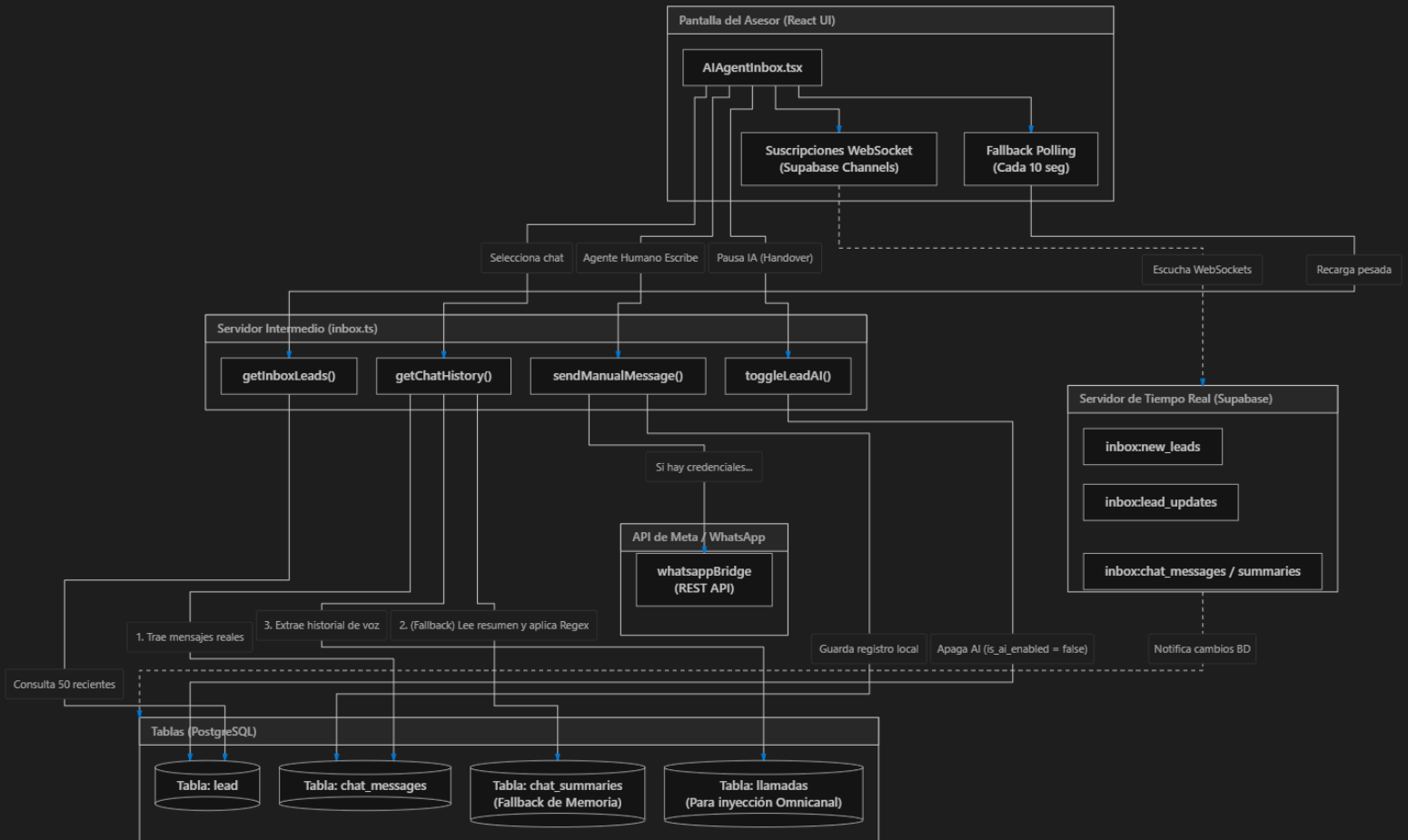
RIESGO DE FORMATO: Reconstrucción Regex de Resúmenes

- **El Problema:** El sistema reconstruye historiales perdidos escaneando el resumen textual buscando el patrón `^[hora] rol: texto`.
- **Impacto:** Si la IA (OpenAI) alucina y genera el resumen usando otro formato (ej. (hora) - usuario - texto), el chat aparecerá completamente en blanco.

4. Resumen para el Cliente (Explicación Sencilla)

1. **Atención Omnicanal en Tiempo Real:** Tu bandeja funciona como el WhatsApp que ya conoces, pero mucho mejor. Los mensajes aparecen al instante sin recargar la página. Además, dentro del mismo chat, podrás ver un aviso si tu Bot de Voz intentó llamar a esa persona 5 minutos antes.
2. **Relevo Automático (Pausa de IA):** Si ves que el bot de texto no está logrando cerrar la venta, simplemente escribes un mensaje. El sistema lo detectará y "apagará" al bot temporalmente para que no se crucen las respuestas y tú puedas tomar el control.
3. 💡 **Optimización Superada (Carga de Datos):** Anteriormente en entornos de prueba, la pantalla se recargaba sola cada 10 segundos para asegurar que no se perdiera ningún

mensaje. Como esto ya fue probado con éxito, ese método pesado ya fue eliminado, logrando que ahora la plataforma funcione 100% de manera fluida y sin consumir datos extra. Queda por pulir un poco la flexibilidad en el envío de plantillas prediseñadas.



Módulo de Calendario y Agendas

Reporte Detallado: Módulo de Calendario y Agendas

Elaborado por: Renzo

Sección Analizada: Calendario & Agendas (/dashboard/calendar)

Este documento detalla la funcionalidad del centro de agendamiento, diseñado no solo para que los asesores humanos vean sus citas, sino para que la **Inteligencia Artificial (Orquestador)** sepa exactamente a qué asesor asignarle la venta basándose en reglas de negocio (Round Robin).

1. Estructura y Funcionamiento en el Panel (La Pantalla)

El panel del calendario está dividido en varias pestañas estratégicas:

Funciones Principales para el Cliente:

- **Agenda Semanal Visual:** Muestra una vista en cuadrícula de lunes a domingo. Las citas se agrupan automáticamente por la hora de España (Europe/Madrid), mostrando visualmente el globo terráqueo si el prospecto tiene otro huso horario local. Si hay dos citas a la misma hora para el mismo asesor, el sistema lanza una alerta visual ("¡CONFLICTO").
- **Matriz de Asesores (El Round Robin):** No es solo una lista de empleados. Cada asesor se configura con "Reglas de Enrutamiento". Puedes asignarle países específicos (ej. "Solo atiende México"), orígenes (ej. "Solo leads de Meta"), y cursos específicos.
- **Bloques de Horarios (Slots):** Permite configurar el horario laboral de cada asesor día por día (ej. Lunes de 09:00 a 18:00) y configurar cuánto dura cada reunión en la empresa (ej. 15 o 30 minutos).
- **Recordatorios Automáticos:** Configuración para decidir si se envían recordatorios por WhatsApp antes de la reunión (ej. 60 minutos antes) y establecer la plantilla de mensaje que se usará.

2. El Mapa Técnico: ¿Qué pasa por detrás?

El sistema de agendamiento es el "puente" entre la Inteligencia Artificial (que cierra la cita por chat/voz) y el humano (que recibe la cita).

A. La Lógica de Enrutamiento (Round Robin)

- La IA revisa los datos capturados del usuario (País, Curso de interés, Campaña) y consulta la tabla advisors.
- Filtra a los asesores que hagan "match" con esos datos.
- Luego llama a la función checkAvailability en la base de datos para ver cuál de esos asesores aptos tiene un hueco disponible en el horario solicitado por el prospecto.

B. Sistema de Citas y Relaciones Fallback

- **Archivo:** src/lib/actions/scheduling.ts
- **Función:** getAppointments()
- **Dato Técnico Interesante:** La base de datos (Supabase PostgREST) a veces falla al tratar de unir directamente la tabla de citas con la tabla de leads debido a limitaciones de caché de esquema. Para evitar esto, el código descarga primero las citas, recolecta todos los IDs de los leads involucrados, y hace una **segunda consulta manual paralela** para traer los nombres y teléfonos de los prospectos y fusionarlos en el servidor (Node.js) antes de mandarlos a la pantalla. Es una forma de "unión por software" (Software Join) muy segura.

C. Sistema de Recordatorios de IA

- Cuando se usa la función createAppointment, el sistema calcula automáticamente en qué momento futuro debería enviarse el recordatorio (restando los minutos configurados, ej. 60 minutos, a la fecha de la cita) y lo guarda en el campo reminder_scheduled_at. El Cron Job general del sistema lee este campo para disparar los mensajes de recordatorio.

D. El Comportamiento de la IA al Agendar (Flujo 100% Funcional)

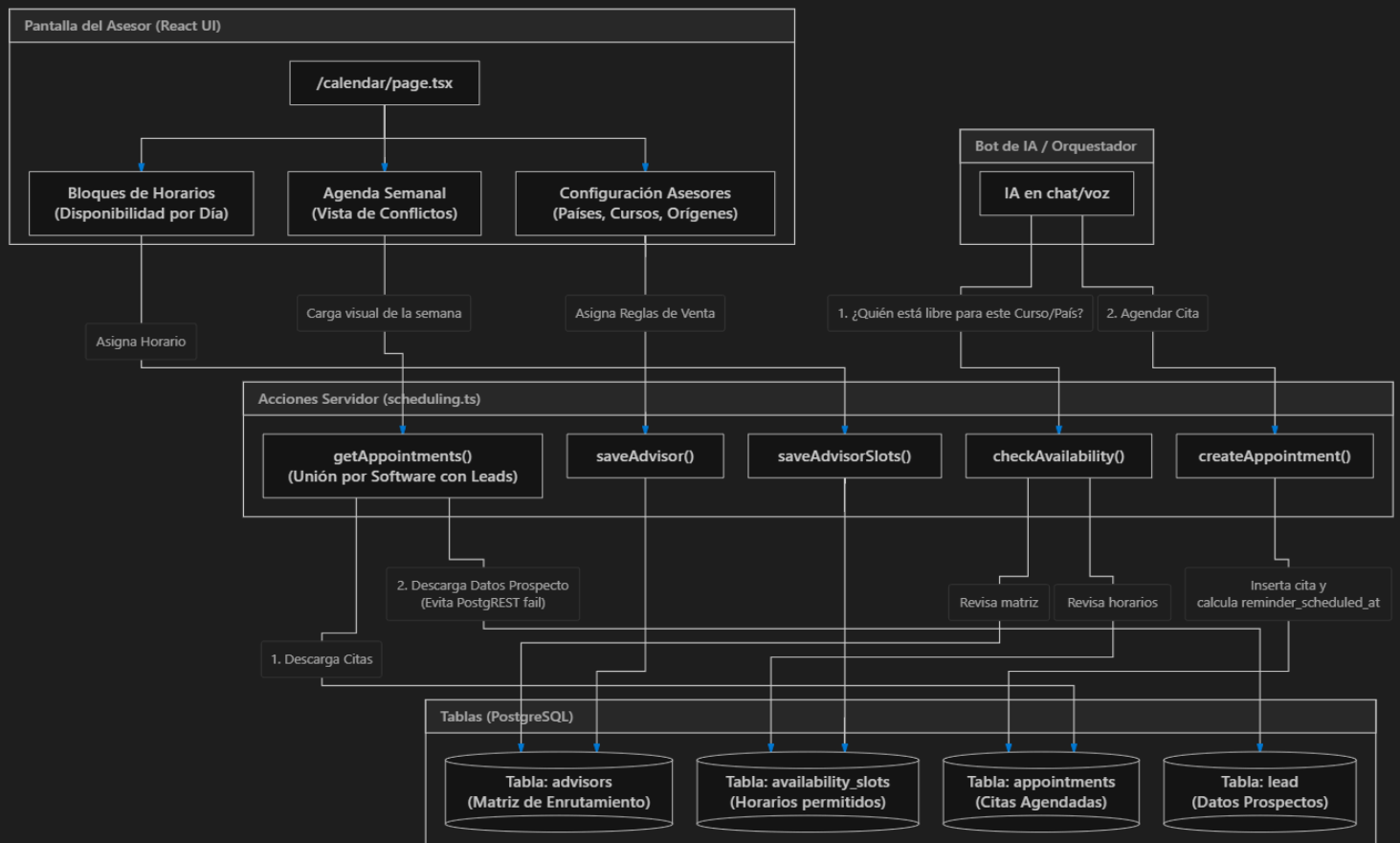
Para entender cómo todo esto se conecta mágicamente, este es el comportamiento exacto de la IA cuando un lead muestra interés. Esta funcionalidad está **totalmente activa y operativa**:

1. **La Conversación Natural:** El Agente de IA (voz o texto) detecta que el prospecto está listo y adopta un comportamiento de asistente ejecutivo, preguntando por sus preferencias: "*¿Te viene bien conversar mañana por la mañana o prefieres por la tarde?*".
2. **La Negociación y Verificación:** El prospecto responde "*Mañana a las 10:00 AM*". La IA extrae esta intención de tiempo y hace un llamado silencioso al backend (checkAvailability). Revisa qué asesor puede atender a ese país/curso (Round Robin) y mira su disponibilidad.
3. **Manejo de Conflictos (Si está ocupado):** Si las 10:00 AM ya están reservadas por otra persona, la IA **no se bloquea**. Su comportamiento es ofrecer alternativas de forma empática: "*Lamentablemente tengo ocupadas las 10:00 AM, pero tengo disponibilidad a las 11:30 AM o el miércoles a las 10:00 AM. ¿Alguna de estas opciones te funciona?*".
4. **Confirmación y Bloqueo Inmediato:** Una vez que el prospecto acepta un horario libre, la IA le dice "*¡Perfecto! Agendado*". En ese mismo milisegundo dispara el comando

createAppointment. La cita aparece dibujada instantáneamente en la pestaña **Agenda Semanal** de la plataforma para el asesor seleccionado, bloqueando el espacio sin intervención humana.

3. Resumen para el Cliente (Explicación Sencilla)

1. **Más que un Calendario, es un Distribuidor Inteligente:** Esta pantalla no es solo para ver a qué hora tienes reunión. Es el "cerebro" que le dice a la Inteligencia Artificial a quién debe asignarle un prospecto. Si configuras a un asesor para que solo atienda "Leads de México que buscan un MBA", la IA jamás le agendará a alguien de España.
2. **Control de Horarios y Conflictos:** Puedes definir el horario de trabajo exacto de cada asesor. Si por error (o si lo fuerzas manualmente) se agendan dos personas en la misma hora exacta para el mismo asesor, el sistema parpadeará en rojo mostrando un aviso de "Conflicto" para que lo soluciones.
3. **Recordatorios Automáticos Integrados:** Puedes configurar aquí mismo que el sistema envíe automáticamente un WhatsApp a tu cliente 1 hora (o el tiempo que decidas) antes de la cita, usando una plantilla personalizada, para reducir las inasistencias.
4. **Agendamiento 100% Autónomo:** El mayor valor de este módulo es que no necesitas operarlo manualmente. Cuando el Bot de Voz o el Chatbot de WhatsApp conversa con el cliente y acuerdan un horario, la IA viene sola a este calendario, revisa quién tiene libre, y planta la cita en la cuadrícula visual de la semana de forma inmediata.



Módulo de Campañas IA

Módulo: Campañas IA

¿Qué es este módulo?

El módulo de **Campañas** es el **centro de mando por iniciativa comercial**. Mientras que el Dashboard General muestra el total de todos los leads, el módulo de Campañas permite enfocar el análisis en **una campaña específica a la vez**: ver cuántos leads entró, cuántos se contactaron, cuántos se cualificaron, y cuántos terminaron agendando una cita.

Piénsalo como el **microscopio** del sistema: el Dashboard es el mapa completo, y Campañas es la lupa que pones sobre una campaña de marketing concreta (Ej: "Google Ads Enero", "Black Friday 2024", "WhatsApp Remarketing Q2").

¿Cómo funciona?

1. Creación de una Campaña

Desde /dashboard/campanas/nuevo, el equipo puede registrar una nueva campaña con:

Campo	Descripción
Nombre	Nombre identificador de la campaña (obligatorio)
Descripción	Texto libre del objetivo
Estado	ACTIVA, PAUSADA, o FINALIZADA
Fecha de Inicio	Desde cuándo corre la campaña
Agente de Chat (WhatsApp)	IA de texto asignada para responder leads de esta campaña
Agente de Voz (Llamadas)	IA de voz asignada para llamar a los leads de esta campaña

 *El punto más importante: al crear la campaña, se le asignan los dos agentes IA (texto y voz). Desde ese momento, cuando un lead entra con el campana = "Nombre Campaña", el sistema sabe automáticamente qué IA debe atenderlo.*

2. Cómo los Leads se Asocian a una Campaña

Los leads no se asignan manualmente a una campaña dentro de la plataforma. La asociación sucede en el momento de ingreso, típicamente desde un formulario web, un anuncio, o un webhook. El campo campana del lead llega como metadato desde la fuente de origen.

```
[Anuncio Google Ads "Enero 2025"]
  → Lead llena formulario
  → Webhook envía al CRM: { nombre, telefono, campana: "Google Ads Enero 2025"
}
  → Sistema registra lead con campana = "Google Ads Enero 2025"
  → La IA del agente asignado a ESA campaña empieza a contactar al lead
```

3. Relación con los Agentes IA

Esta es la conexión más importante del módulo:

Campaña "Google Ads Enero 2025"

```
├─ agente_texto_id → Agente "Asistente WhatsApp Pro"
│   └─ Cuando el lead responde por WhatsApp, este agente toma la
conversación
│       usando su prompt, knowledge base, y reglas de cualificación
├─ agente_llamada_id → Agente "Llamador Formación España"
│   └─ Cuando el dialer llama al lead, este agente conduce la llamada de
voz
│       usando ElevenLabs (síntesis de voz) + GPT (inteligencia
conversacional)
```

En resumen: la campaña es el "contenedor" que une leads → agentes IA → métricas. Sin campaña asignada, el lead existe en el sistema pero sin contexto de rendimiento.

4. Panel de Métricas (KPIs Hero)

Al entrar a /dashboard/campanas, la primera sección muestra 7 cifras en tiempo real:

Métrica	Qué mide
Total Leads	Todos los leads con esa campaña en el período
Llamadas	Intentos de contacto realizados por la IA de voz
Contactados	Leads que realmente contestaron (estado llamada = "CONTACTED")
Agendados	Leads que terminaron con una cita en el calendario
Cualificados	Leads que la IA marcó como interesados/aptos
Minutos IA	Total de minutos que la IA habló (costo operativo real)
Nº Campañas	Cuántas campañas distintas hay en el período

5. El Filtro de Campaña

La barra superior tiene un selector de campaña (CampaignSelector). Al seleccionar una campaña específica (Ej: "Black Friday 2024"), todos los KPIs, gráficos y el embudo se filtran automáticamente para mostrar solo datos de esa campaña. Esto permite comparar el rendimiento de diferentes campañas de forma aislada.

6. Gráficos de Análisis

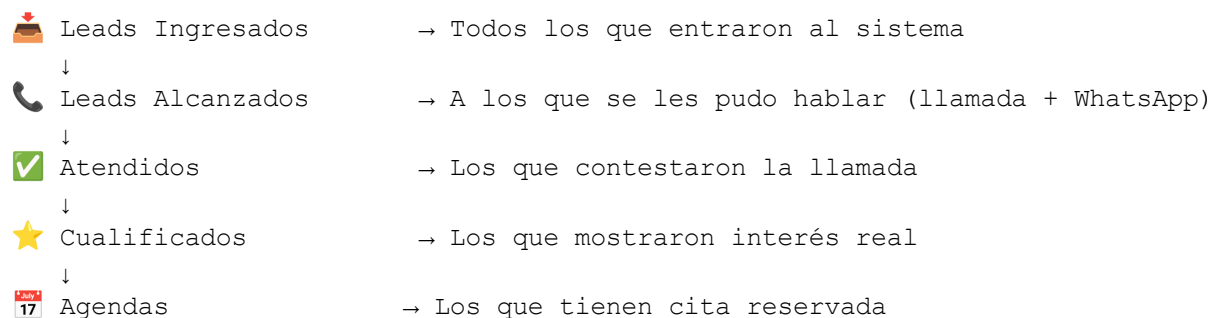
Hay 4 gráficos por defecto (configurables por el administrador):

Gráfico	Tipo	Qué muestra
---------	------	-------------

Leads por Fecha	Área	Evolución temporal de entrada de leads
Leads por Origen	Barras	De dónde vienen los leads (Google, Facebook, etc.)
Tipo de Lead	Donut	Distribución: Interesado / Ilocalizable / Rechazado
Primer Contacto por Fecha	Área	Cuándo la IA hizo el primer contacto

7. El Embudo de Conversión

El embudo muestra el recorrido completo de un lead dentro de la campaña:



Cada etapa muestra el número absoluto y la tasa de conversión respecto a la anterior, lo que permite identificar en qué punto se pierde más tráfico.

8. Tabla de Prospectos Recientes

Al final de la página hay una tabla con los últimos 10 leads de la campaña filtrada, mostrando: nombre, apellido, teléfono, curso de interés, origen y fecha de ingreso. Desde esta tabla también se puede crear un nuevo lead manual (botón "+ Nuevo Lead").

Cómo interpreta los datos el sistema

El sistema no hace JOINS directos en la base de datos entre lead y las tablas de actividad. En cambio:

1. Carga todos los leads del tenant (en páginas de 1.000) y los guarda en memoria como un Map<lead_id, lead>.
2. Carga las tablas de hechos (llamadas, agendamientos, cualificaciones, WhatsApp) de forma paralela.
3. Cruza los datos en el servidor usando el Map para vincular cada actividad a su campaña.
4. Agrupa por nombre de campaña y calcula las métricas derivadas.

Esto evita errores de cache de esquema de Supabase y da control total sobre los filtros multi-nivel.

Resumen para el Cliente

El módulo de Campañas te permite ver, campaña por campaña, cuánto rindió cada inversión de marketing. En lugar de ver todos los leads mezclados, filtras por "Google Ads Enero" y ves exactamente cuántos leads captaste, cuántos contestaron el teléfono, cuántos quedaron

convencidos, y cuántos agendaron una cita.

Al crear una campaña, también le asignas los agentes IA (texto y voz) que la atenderán. Así, cada lead que entra por esa campaña es atendido automáticamente por la IA correcta, sin que nadie tenga que asignar nada manualmente.

El Embudo de Conversión te dice en qué punto exacto pierdes más prospectos, para que puedas ajustar el mensaje o la estrategia.

Estado del Módulo: **Funcional y Operativo**

- Creación de campañas:  Funcional (INSERT en tabla campanas)
- Asignación de agentes IA:  Funcional (campos agente_texto_id / agente_llamada_id)
- KPIs en tiempo real:  Funcional (7 métricas hero + cards configurables)
- Gráficos dinámicos:  Funcional (4 gráficos configurables por admin)
- Embudo de conversión:  Funcional (5 etapas con tasas)
- Filtro por campaña:  Funcional (URL param ?campana=...)
- Tabla de prospectos:  Funcional (últimos 10 leads de la campaña)
- Configuración por admin:  Funcional (drag & drop KPIs y gráficos)

Mapa Técnico — Módulo de Campañas

Mapa Técnico — Módulo de Campañas

Árbol de Archivos

```
src/app/dashboard/campanas/  
├─ page.tsx           ← Página principal (Server Component)  
└─ nuevo/  
    └─ page.tsx       ← Formulario de creación de campaña (Client  
Component)  
  
src/lib/actions/  
├─ campanas.ts        ← Server Actions: createCampaign(), getCampaigns()  
└─ analytics.ts       ← getKpiCampanas(), getUniqueCampaigns(),  
getDynamicChartSeries()  
  
src/lib/constants/  
└─ kpi-defaults.ts    ← CAMPANAS_KPIS, CAMPANAS_CHARTS, CAMPANAS_FUNNEL  
  
src/components/campanas/  
└─ CampaignLeadsTable.tsx ← Tabla de prospectos recientes por campaña  
  
src/components/dashboard/  
├─ SummaryManager.tsx ← Motor de tarjetas KPI (configurable por admin)  
├─ ChartManager.tsx   ← Motor de gráficos (configurable por admin)  
├─ FilterBar.tsx      ← Barra de filtros: fecha, país, origen, campaña  
└─ CampaignSelector.tsx ← Selector de campaña activa (URL param)
```

Interfaces de Datos Clave

KpiCampanas (analytics.ts)

```
interface KpiCampanas {  
  campanas: CampanaRow[]; // Array con stats por cada campaña  
  leads_por_campana: ChartRow[]; // Para gráficos de barras  
  contactados_por_campana: ChartRow[];  
  cualif_por_campana: ChartRow[];  
  agendados_por_campana: ChartRow[];  
  minutos_por_campana: ChartRow[];  
  total_leads: number;  
  total_llamadas: number;  
  total_contactados: number;  
  total_agendados: number;  
  total_cualificados: number;  
  total_minutos: number;  
  total_segundos: number;  
  total_leads_alcanzados: number;  
  leads_por_dia: ChartRow[];  
}
```

CampanaRow (analytics.ts)

```
interface CampanaRow {  
  nombre: string;  
  total_leads: number;  
}
```

```

    contactados: number;
    no_contacto: number;
    tasa_contacto: number;          // Calculado: (contactados/total_llamadas)*100
    cualificados: number;
    no_cualificados: number;
    agendados: number;
    total_llamadas: number;
    total_minutos: number;
    total_segundos: number;
    duracion_media_seg: number;     // Calculado: total_segundos/total_llamadas
}

```

Flujo de Datos — Página Principal

URL: /dashboard/campanas?from=...&to=...&campana=...

CampanasPage (Server)

```

├─ parseFilters(searchParams)      → { from, to, filters }
├─ getUniqueCampaigns()            → string[] (nombres de campañas)
├─ getAdminStatus()                → boolean
├─ <Suspense> → CampanasKpis
│   ├─ getActiveTenantConfig()      → config del tenant (kpis_campanas)
│   ├─ getKpiCampanas(from, to, filters)
│   └─ getDynamicKpis(...)          → KPIs personalizados del admin
│       └─ <SummaryManager layout="cards" configKey="kpis_campanas" />
│           └─ HeroStat[] → Total Leads / Llamadas / Contactados / Agendados /
Cualificados / Minutos IA / N° Campañas

```

```

├─ <Suspense> → CampanasChartsSection
│   ├─ getActiveTenantConfig()      → config del tenant (charts_campanas)
│   └─ getDynamicChartSeries(charts, from, to, filters)
│       └─ <ChartManager configKey="charts_campanas" />
│           ├─ Area: Leads por Fecha      (lead.fecha_ingreso_crm)
│           ├─ Bar: Leads por Origen      (lead.origen)
│           ├─ Donut: Tipo de Lead        (lead.tipo_lead)
│           └─ Area: Primer Contacto/Fecha (llamadas.fecha_inicio)
├─ <Suspense> → CampanasFunnel
│   ├─ getKpiCampanas(from, to, filters)
│   └─ <SummaryManager layout="funnel" configKey="funnel_campanas" />
│       └─ Leads → Alcanzados → Atendidos → Cualificados → Agendados
├─ <Suspense> → CampanasLeadsSection
│   ├─ fetchCalls({ page:1, campana, pageSize:10 })
│   └─ <CampaignLeadsTable data total />
│       └─ Tabla: Nombre | Apellido | Teléfono | Curso | Origen | Campaña |
Fecha

```

Algoritmo getKpiCampanas

Patrón: Manual Join en memoria (Hub & Spoke)

```

PASO 1 – Paginación de Leads (chunks de 1000)
├─ SELECT id, campana, pais, origen, tipo_lead
  FROM lead WHERE tenant_id = ? [+ filtros]

```

```
→ leadsMap: Map<lead_id, lead>
```

PASO 2 – Fetch paralelo de tablas de hechos

```
| llamadas → id, id_lead, estado_llamada, duracion_segundos
| agendamientos → id, id_lead (WHERE confirmado = true)
| lead_cualificacion → id, id_lead, cualificacion
| conversaciones_whatsapp → id_lead
```

PASO 3 – Agrupación por campaña

```
| campMap[nombre_campana] = { total_leads, contactados, agendados, ... }
|
| Leads: allLeads.filter(l => l.campana).forEach →
campMap[l.campana].total_leads++
| Llamadas: se vinculan vía leadsMap.get(id_lead) →
campMap[lead.campana].total_llamadas++
| Agendamientos: igual, → campMap[lead.campana].agendados++
| Cualificaciones: → campMap[lead.campana].cualificados++ (si != "NO")
```

PASO 4 – total_leads_alcanzados

```
└ reachedSet = Set de lead_ids que tuvieron llamada OR whatsapp
```

PASO 5 – Cálculos derivados por campaña

```
└ tasa_contacto = (contactados / total_llamadas) * 100
└ duracion_media_seg = total_segundos / total_llamadas
```

PASO 6 – Retorno

```
└ Totales globales reducidos + arrays para gráficos
```

Tabla campanas en Supabase

Campo	Tipo	Descripción
id	UUID	PK
tenant_id	UUID	Aislamiento multi-tenant
nombre	text	Nombre de la campaña
descripcion	text	Descripción libre
estado	text	ACTIVA / PAUSADA / FINALIZADA
fecha_inicio	date	Inicio de la campaña
fecha_fin	date	Fin (nullable)
agente_texto_id	UUID	FK → ai_agents (WhatsApp)
agente_llamada_id	UUID	FK → ai_agents (Voz)
fecha_creacion	timestamp	Auto

KPIs y Gráficos por Defecto

KPIs (CAMPANAS_KPIS)

id	Label	staticKey	Color
cp-1	Total Leads	total_leads	bg-blue-600
cp-2	Contactados	total_contactados	bg-emerald-600
cp-3	Cualificados	total_cualificados	bg-yellow-600
cp-4	Agendados	total_agendados	bg-teal-600

cp-5	No Cualificados	total_no_cualificados	bg-rose-600
cp-6	T. Respuesta	total_segundos	bg-amber-600

Gráficos (CAMPANAS_CHARTS)

id	Tipo	Título	Fuente xKey
cp-ch-1	area	Leads por Fecha	lead.fecha_ingreso_crm
cp-ch-2	vertical-bar	Leads por Origen	lead.origen
cp-ch-3	donut	Tipo de Lead	lead.tipo lead
cp-ch-4	area	Primer Contacto/Fecha	llamadas.fecha_inicio

Módulo: Llamadas y Minutos (Agente de Voz)

Módulo: Llamadas y Minutos (Agente de Voz)

¿Qué es este módulo?

Este módulo constituye el núcleo de control e inteligencia del Agente de Voz automatizado dentro de la plataforma. Su arquitectura se divide en dos secciones principales e independientes dentro del dashboard:

1. **Métricas de Llamadas (/dashboard/minutos):** El panel analítico estructurado para medir el rendimiento, consumo financiero y conversión del Agente de Voz automatizado.
2. **Lanzador Manual (/dashboard/calls):** Una interfaz operativa premium tipo "Dialer" diseñada para que los operadores o administradores lancen llamadas proactivas puntuales usando la IA, observando la transcripción en tiempo real.

1. Panel de Métricas (Minutos / Agente de Voz)

Ruta en repositorio: /dashboard/minutos/page.tsx

Estado Funcional:  OPERATIVO / LISTO PARA PRODUCCIÓN

Este panel se enfoca puramente en el rendimiento del bot de voz. Al igual que otros módulos analíticos de la plataforma, se basa en la infraestructura dinámica de los componentes SummaryManager y ChartManager para permitir configuraciones en tiempo real.

KPIs Principales (Hero Metrics)

Las métricas críticas se calculan de forma correcta y automatizada en la función getKpiMinutos del archivo analytics.ts:

- **Total de Minutos:** Sumatoria de la duración de todas las llamadas realizadas. Es una métrica crítica para el control de costos operativos y la auditoría de facturación de proveedores externos (ej. Retell / ElevenLabs).
- **Duración Media:** Promedio de segundos transcurridos por cada llamada.
- **Llamadas Realizadas:** Cantidad bruta total de intentos de llamadas disparados por el sistema.
- **Contactadas:** Cantidad de llamadas donde hubo una respuesta humana efectiva, identificada bajo el parámetro estado_llamada = 'CONTACTED'.

Métricas Adicionales (Configurables)

El sistema aplica correctamente los filtros globales de fecha y los atributos específicos del lead sobre las siguientes variables:

- Tasa de Agendamiento (%) y Tasa de Conversión (%).
- Gráficos de distribución de llamadas ordenados por fecha, por estado final (Contactado, Buzón de voz, Sin respuesta) y por razón explícita de término de la comunicación.

2. Lanzador Manual de Llamadas (Retell AI SDK)

Ruta en repositorio: /dashboard/calls/page.tsx

Estado Funcional: ⚠️ PROTOTIPO VISUAL (MOCKUP)
Una interfaz premium (versión v2.0) diseñada con el objetivo de permitir que un agente humano inicie una llamada saliente de la IA hacia un número de teléfono específico de forma proactiva.

Funcionalidades Diseñadas (Interfaz de Usuario)

- **Dialer (Teclado Numérico):** Interfaz visual interactiva para ingresar el número telefónico de destino.
- **Pre-Call Checklist:** Verificación visual previa del horario legal de contacto y el saldo disponible de créditos antes de lanzar el proceso.
- **Live Monitor:** Un panel interactivo configurado para mostrar de forma simulada la transcripción de texto en tiempo real de la conversación activa entre el Lead y la IA.
- **Historial Reciente:** Lista de acceso rápido que muestra las últimas llamadas ejecutadas desde este componente.

Errores Técnicos y Elementos Faltantes (Mocks)

Actualmente, esta sección de la plataforma es **exclusivamente visual (UI) y está mockeada**, por lo que carece de lógica de negocio en el backend:

- **Llamada simulada:** La función principal handleCall no se conecta con ningún servicio o backend real. Utiliza un temporizador estático (setTimeout de 5 segundos) para simular que la llamada se encuentra en progreso mediante el flag setIsCalling.
- **Transcripción Falsa:** Todo el texto de la conversación visible ("Hola, soy el asistente...", "Sí, con él habla...") se encuentra hardcoded dentro del componente HTML, sin flujos reales de WebSockets del SDK de Retell.
- **Lista Hardcodeada:** La sección correspondiente a "Llamadas Recientes" despliega datos estáticos de prueba (ej. "+34 612 345 678").

3. Resumen Técnico de Extracción de Datos

Las métricas analíticas se obtienen mediante el cruce relacional entre la tabla llamadas y la tabla lead en la base de datos Supabase. Cada fila de la tabla llamadas representa un intento de salida del sistema e incluye los siguientes campos clave:

Campo (Columna)	Descripción y Uso Técnico
estado_llamada	Registra el resultado de la conexión (CONTACTED, NO_CONTACT, VOICEMAIL, etc.).
duracion_segundos	Almacena los segundos reales consumidos. Fundamental para la métrica principal y el control de costos.
razon_termino	Indica el motivo de finalización (USER_INTERRUPTED, AGENT_HANGUP, etc.).
url_grabacion	Enlace de almacenamiento directo hacia el

Campo (Columna)	Descripción y Uso Técnico
	archivo de audio (.mp3/.wav) de la llamada.

Conclusión y Próximos Pasos (Roadmap de Desarrollo)

El **Panel Analítico de Minutos** se encuentra al **100% funcional** y en condiciones óptimas para su despliegue definitivo en producción. Por el contrario, el **Lanzador Manual (/calls)** requiere desarrollo técnico en el backend para pasar de un estado de prototipo a uno operativo. Para su habilitación real, se definen las siguientes prioridades:

1. Crear un endpoint intermedio estructurado bajo la ruta `/api/calls/manual` que reciba la petición e invoque de forma directa la creación de la llamada en las plataformas de Retell o Vapi.
2. Implementar un canal de comunicación persistente por WebSockets (o esquemas rápidos de polling) en el frontend con el fin de capturar de forma síncrona los eventos de transcripción en vivo emitidos por el motor de IA, renderizándolos dinámicamente en el componente Live Monitor.

Mapa Técnico — Módulo Llamadas (Voz)

Mapa Técnico — Módulo Llamadas (Voz)

1. Árbol de Archivos y Estructura del Módulo

A continuación se detalla la ubicación y propósito de cada archivo involucrado en la lógica y renderizado del módulo de voz:

```
src/app/dashboard/
├── minutos/
│   └── page.tsx                ← Dashboard Analítico (Agente de
Voz)
└── calls/
    └── page.tsx                ← Marcador Manual (Retell SDK
Dialer) - Mockup

src/lib/actions/
└── analytics.ts                ← getKpiMinutos(),
getDynamicChartSeries()

src/lib/constants/
└── kpi-defaults.ts            ← LLAMADAS_KPIS,
DEFAULT_CHARTS_LLAMADAS
```

2. Diagrama de Flujo: Panel de Minutos (Analítica)

El panel analítico procesa y carga las métricas de negocio desde la tabla de hechos llamadas en Supabase, la cual actúa como registro maestro de cada sesión de Retell o Vapi. El orden de ejecución y flujo de datos sigue este camino:

1. El componente **Dashboard Minutos** envía los filtros de fecha y atributos activos a la función de servidor `getKpiMinutos`.
2. La función ejecuta un `SELECT` sobre la tabla llamadas en **Supabase**, extrayendo campos clave como `id`, `duracion_segundos` y `estado_llamada`.
3. Se realiza el **Cálculo de Agregados** en el servidor:
 - Sumatoria de segundos reales para `total_segundos`.
 - Conteo bruto de registros para `total_llamadas`.
 - Cálculo de promedios para determinar la `duracion_media`.
4. La respuesta con el objeto estructurado `KpiMinutos` se envía al cliente, donde el componente `SummaryManager` se encarga de renderizar las Hero Metrics en la UI.
5. En paralelo, el dashboard invoca a la función `getDynamicChartSeries` para agrupar los datos por rango temporal y pasárselos al componente `ChartManager` para pintar los gráficos analíticos.

3. Diagrama Estructural: Lanzador Manual (/calls)

La sección del marcador es actualmente un **Prototipo UI (Mockup)** que emula el ciclo de vida de una llamada real en el componente frontend `CallsPage` mediante estados locales:

- **Estado Inicial (Standby):** La interfaz espera una interacción. Al tipear en el teclado

- numérico (Dialpad), pasa al estado *TypingNumber*.
- **Marcado (Calling):** Al presionar el botón de llamada, se ejecuta la acción simulada, cambiando el flag `setIsCalling(true)` y activando un temporizador automático (`setTimeout`) de 5000 milisegundos (5 segundos).
 - **Monitoreo en Vivo (StreamingUI):** Mientras dura el temporizador, la UI condicional se renderiza mostrando:
 - Un encabezado de "Live Monitor" con animación intermitente en verde (Pulse).
 - Burbujas de chat de la conversación totalmente hardcodeadas en el código.
 - El botón de acción con estado animado "En llamada..."
 - **Fin de Interacción:** Una vez transcurridos los 5 segundos, el temporizador expira y la interfaz regresa automáticamente al estado de *Standby*.

Código Fuente del Mockup

Ubicación exacta del fragmento de simulación en el archivo `src/app/dashboard/calls/page.tsx` (Líneas 27 a 31):

```
const handleCall = async () => {  
  setIsCalling(true);  
  // Simulate API call to /api/calls/manual  
  setTimeout(() => setIsCalling(false), 5000); // Simulate end  
};
```

4. Próximos Pasos para Integración Real

Para sustituir este flujo mockeado por una conexión real con el SDK de Retell/Vapi, es necesario sustituir la lógica del temporizador por una petición HTTP real hacia un endpoint de servidor que inicialice la llamada, seguido de la apertura de un canal WebSocket para escuchar la transcripción en vivo bit a bit.

Módulo: Historial de Llamadas y Leads

Módulo: Historial de Llamadas y Leads

¿Qué es este módulo?

El módulo de **Historial** (/dashboard/historial) es la tabla maestra (CRM) de la plataforma. A diferencia de los otros módulos analíticos que se enfocan en agrupar métricas acumuladas (KPIs y Gráficos), el Historial despliega el **detalle fila por fila de cada Lead** y refleja su estado más reciente dentro del flujo de ventas de la empresa.

Constituye el espacio operativo principal donde los administradores y asesores comerciales auditan a nivel individual el recorrido de un prospecto: si fue contactado, si posee conversaciones activas en WhatsApp, qué respuestas entregó y qué agente de IA asistió la interacción.

Estructura y Componentes Operativos

1. Tabla Dinámica de Prospectos (HistorialTable)

- **Carga Eficiente:** Implementa paginación gestionada directamente en el servidor (SSR) mediante la función de backend `fetchCalls`, sirviendo los datos en bloques optimizados de 50 resultados.
- **Visualización de Datos:** Muestra los leads ordenados por ingresos recientes, indicando país de procedencia, campaña de marketing de origen, la última cualificación registrada y el estado final de su interacción más reciente.
- **Filtros Avanzados superiores:** Permite segmentar el universo de registros mediante búsquedas directas por nombre o número telefónico, rangos de tiempo preestablecidos (Hoy, Ayer, Últimos 7 días) y selectores específicos para el Estado de la Llamada, Tipo de Lead y Nivel de Cualificación.
- **Acciones Manuales:** Integra el botón "Nuevo Lead" que acciona el componente modular `CreateLeadDialog`, permitiendo la inserción manual de prospectos directo al CRM sin depender de automatizaciones de entrada.

2. Panel de Trazabilidad Omnicanal (LeadTraceabilitySidebar)

- **Interfaz Flotante:** Al seleccionar cualquier registro de la tabla principal, el sistema despliega un panel lateral modal flotante interactivo.
- **Metadatos del Prospecto:** Concentra información clave del perfil como país de residencia, datos de contacto y el programa o producto de interés del cliente.
- **Línea de Tiempo Dinámica (Timeline):** Invoca de forma asíncrona a la función `fetchLeadEvents` para reconstruir cronológicamente el historial guardado en Supabase, detallando hitos críticos:
 - Fecha y hora de inserción al CRM.
 - Logs de envíos de notificaciones o mensajes automatizados disparados por la IA.
 - Resultados detallados de los intentos de llamada realizados por el bot de voz.

3. Gestor de Columnas de Administrador (HistorialColumnManager)

- **Acceso Restringido:** Herramienta exclusiva disponible únicamente para cuentas de

- usuario autenticadas con rol de ADMIN, visible en la franja inferior del historial.
- **Tecnología Drag & Drop:** Desarrollado sobre la librería comercial @dnd-kit/core, otorga al administrador la capacidad de:
 - Ocultar o incorporar columnas de forma interactiva a la vista del historial.
 - Alterar la prioridad posicional y el ordenamiento visual arrastrando los bloques de cabecera.
 - Renombrar las etiquetas comerciales de los encabezados de la tabla para adaptarlas al glosario de la empresa.
- **Persistencia Multi-tenant:** Las configuraciones personalizadas se almacenan de manera aislada en el perfil config del Tenant correspondiente mediante la ejecución de la función updateTenant.

Resumen Técnico y Lógica de Base de Datos

La función central fetchCalls() (ubicada en actions/calls.ts) efectúa un procesamiento pesado en el servidor para unificar los datos dispersos y consolidar las filas del historial. El proceso se ejecuta en dos fases secuenciales:

1. Consulta Base (Supabase): Realiza una petición anidada estructurada para capturar la entidad lead con sus relaciones directas de llamadas y cualificaciones:

```
supabase.from("lead").select(`
  *,
  llamadas (...),
  lead_cualificacion (...)
`)
```

2. Consultas Cruzadas en Memoria: Con los IDs obtenidos en la primera fase, ejecuta consultas secundarias acotadas para extraer las entidades de appointments (citras agendadas) y conversaciones_whatsapp, evitando sobrecargar un único Query con joins masivos. Finalmente, compila toda la información en un objeto plano de TypeScript, deduciendo en tiempo de ejecución las variables latestCall, latestCual y latestAppt para asegurar que el usuario visualice los estados más frescos.

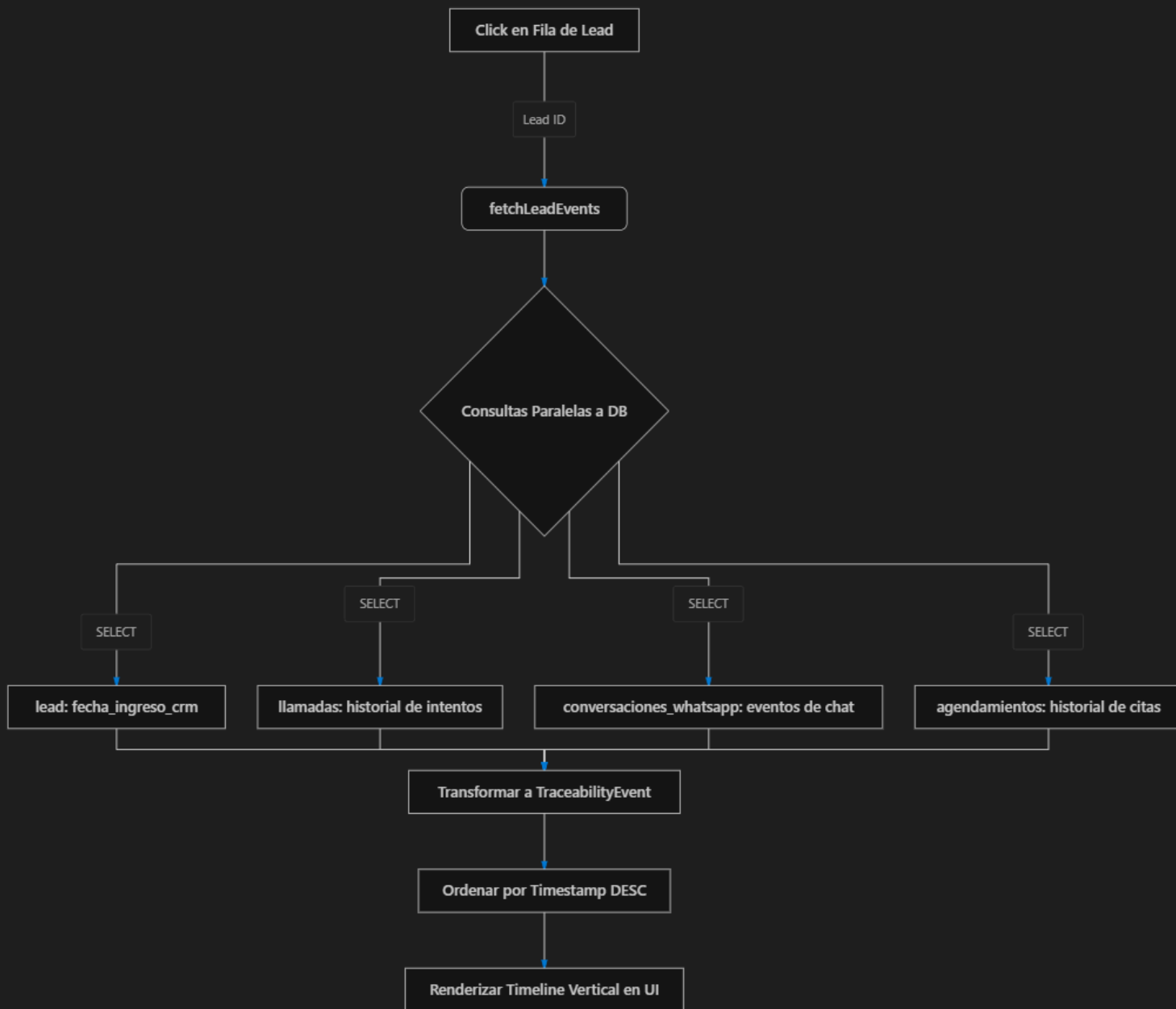
Hallazgos, Riesgos y Áreas de Mejora

Área Identificada	Detalle Técnico y Recomendación
Rendimiento de Paginación (Escalabilidad)	El uso simultáneo de range(from, to) acoplado a subconsultas anidadas y filtros lógicos .filter aplicados en la memoria del servidor puede degradar drásticamente los tiempos de respuesta. Este esquema generará un cuello de botella si el volumen de la tabla supera los 100,000 registros. <i>Recomendación:</i> Desarrollar una Vista Materializada (Materialized View) o tablas de agregación en PostgreSQL que mantengan indexado y precalculado el estado más reciente de cada lead de forma nativa.
Características Inactivas (Código)	El archivo fuente de HistorialTable.tsx mantiene inhabilitado por comentarios el import del reproductor de audio: // import {

Área Identificada	Detalle Técnico y Recomendación
Comentado)	AudioPlayer } from "@components/historial/AudioPlayer";. Esto evidencia que la plataforma contemplaba la reproducción nativa de las grabaciones desde la tabla principal, pero actualmente se encuentra desactivada.
Control de Duplicados	La aplicación cuenta con el componente DuplicateLeadDialog, pero su lógica se ejecuta de manera reactiva (ad-hoc) únicamente al intentar crear o abrir registros con teléfonos preexistentes. Esto puede causar inconsistencias menores en métricas exactas de atribución de campañas.

Estado General del Módulo

El módulo se encuentra  **plenamente funcional, operativo y en un estado visualmente avanzado**. Destaca la calidad técnica implementada en la trazabilidad del sidebar y la versatilidad del gestor Drag & Drop de columnas para los administradores. Los puntos críticos reportados corresponden a tareas de mantenimiento preventivo y optimización de base de datos de cara al crecimiento del volumen de leads comerciales.



Módulo: WhatsApp Leads

Reporte del Módulo: WhatsApp Leads

¿Qué es este módulo?

El módulo de **WhatsApp** (/dashboard/whatsapp) está centrado exclusivamente en medir el volumen, la eficacia y la conversión generada a través de las interacciones de texto automatizadas por el agente de Inteligencia Artificial en WhatsApp.

Este panel permite auditar el comportamiento del bot de chat a lo largo del tiempo, evaluar la densidad de conversaciones simultáneas que sostiene y cuantificar a cuántos de esos prospectos logra guiar exitosamente hacia el agendamiento de una cita.

1. Panel de Métricas Analíticas

El módulo reutiliza la arquitectura dinámica basada en los componentes SummaryManager y ChartManager para renderizar los datos consolidados y procesados por la función de servidor getKpiWhatsapp().

KPIs Principales (Hero Metrics)

- **Conversaciones:** Sumatoria total de mensajes e hilos de chat procesados de forma automatizada por el sistema.
- **Leads Únicos:** Cantidad neta de personas distintas (identificadores únicos de la tabla lead) que mantuvieron al menos una interacción por el canal de WhatsApp.

Métricas Adicionales Configurables

Aplicando en tiempo real los filtros globales de temporalidad y los atributos del lead, el sistema calcula:

- **Agendados / % Agenda:** Volumen y porcentaje de leads que interactuaron por WhatsApp y concluyeron el flujo con una cita formal programada.
- **Cualificados / % Conversión:** Leads detectados y marcados como aptos tras la auditoría automática de su conversación en el chat.
- **% Ilocalizables:** Tasa de leads que, a pesar de haber registrado interacción inicial, fueron clasificados posteriormente como registros perdidos.
- **Gráficos Dinámicos de Apoyo:** Curvas de conversaciones desglosadas por día, segmentación por el origen/campaña de los leads de chat y gráficos de pastel con el estado de la cualificación.

2. Resumen Técnico: El "Manual Join" en Memoria

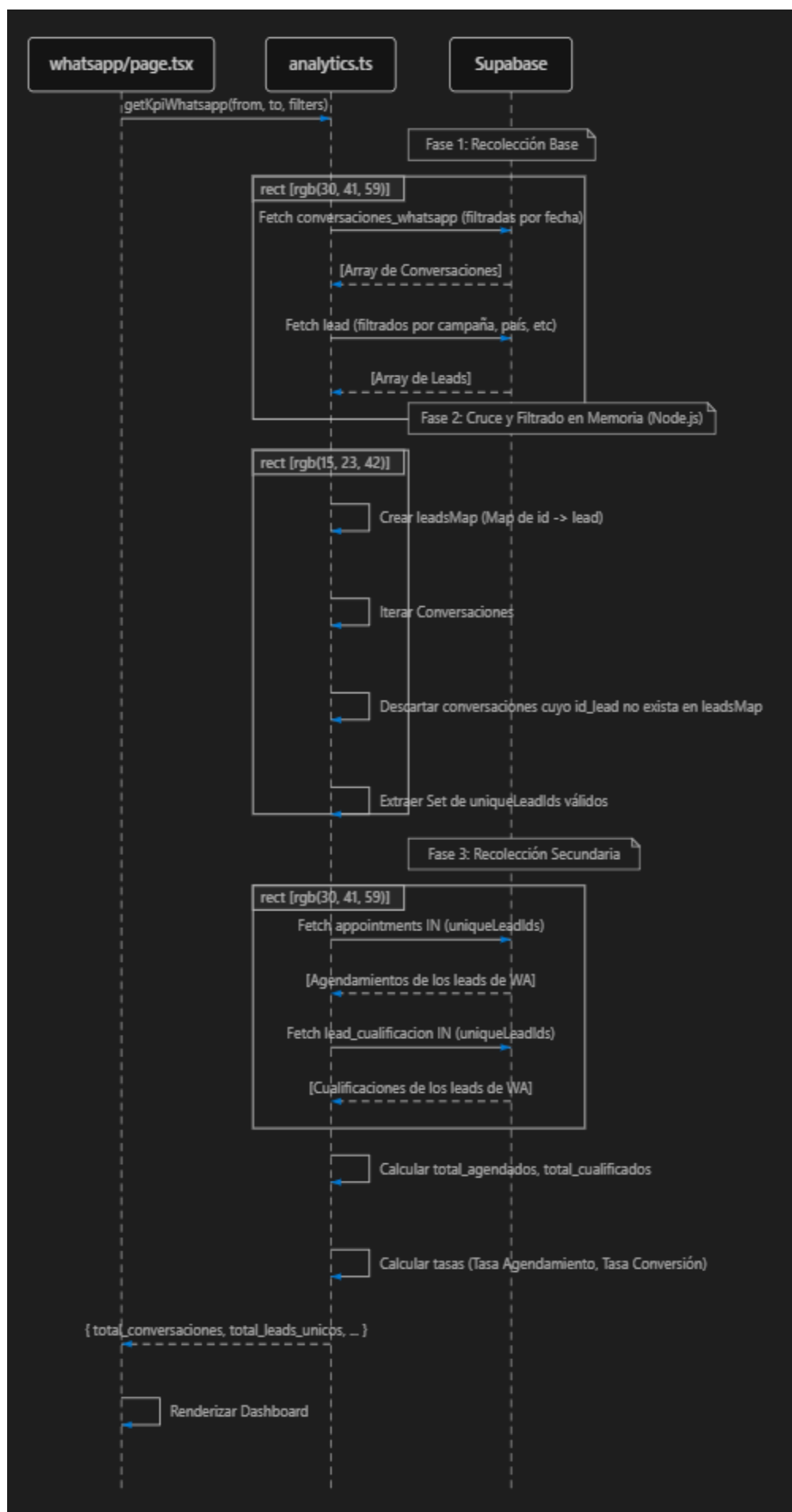
Al igual que en el módulo de campañas, la extracción y procesamiento de datos de WhatsApp en analytics.ts (específicamente en la función getKpiWhatsapp) **evita realizar JOINS complejos a nivel de base de datos en Supabase**. Esta decisión de diseño previene fallos potenciales relacionados con la caché de esquemas anidados y optimiza el comportamiento de las políticas de RLS (Row Level Security).

El algoritmo de consolidación opera bajo el siguiente flujo secuencial en el servidor:

Paso	Acción en Servidor	Propósito Técnico
1	Descarga total de conversaciones_whatsapp	Extrae los registros correspondientes al Tenant activo, acotados por la fecha de registro del último mensaje.
2	Descarga en paralelo de la tabla lead	Trae los leads que coinciden de forma estricta con los filtros globales de la UI.
3	Cruce indexado en memoria de Node.js	Valida las relaciones mapeando las entidades mediante un diccionario estructurado (leadsMap.has(r.id_lead)).
4	Consultas secundarias acotadas	Con la lista reducida de IDs de leads válidos filtrados en el paso anterior, se consultan las tablas de agendamientos y lead_cualificacion.
5	Agregación y cálculo de tasas	Se computan los totales finales y se procesan matemáticamente los porcentajes de agenda y conversión.

Conclusión del Estado Actual

El **Panel Analítico de WhatsApp** se encuentra  **plenamente operativo y validado**. El mecanismo de cruce de datos en la capa del servidor (Node.js) garantiza la precisión en las métricas de conversión y previene la degradación de tiempos de respuesta provocada por consultas relacionales masivas en la base de datos.



simuladores

Módulo: Orchestrator Playground

Reporte del Módulo: Orchestrator Playground

¿Qué es este módulo?

El **Orchestrator Playground** (/dashboard/playground) es un entorno de simulación en vivo y diagnóstico avanzado diseñado para que los administradores de la plataforma puedan evaluar el comportamiento de un flujo de trabajo (Workflow) sobre un registro de Lead real seleccionado de la base de datos. Su propósito principal es auditar de manera secuencial y transparente las acciones, ramificaciones y decisiones lógicas que el motor de orquestación central ejecuta paso a paso.

Estructura y Flujo de Operación

La arquitectura del Playground está distribuida estratégicamente para ofrecer un diagnóstico modular e integral:

Componente / Panel	Propósito Operativo y Funcionalidad Técnica
1. Dashboard de Diagnóstico	Se ejecuta de manera automática al cargar el módulo. Realiza un checklist de infraestructura para verificar que el Tenant cuente con las credenciales esenciales activas: Token de Meta/WhatsApp, configuraciones de AWS Bedrock Knowledge Bases (KB) y el estado general de conexión con la Base de Datos.
2. Panel de Setup (Izquierdo)	Concentra los selectores de control del entorno. Permite al usuario elegir un Workflow específico y asociarlo a un Lead real del sistema. Al hacerlo, despliega dinámicamente el listado de reglas y pasos lógicos que componen dicho flujo de automatización.

3. Panel de Ejecución (Derecho)	Monitorea la actividad del proceso mediante dos pestañas de visualización avanzada: • Consola en Vivo (Live Console): Imprime en tiempo de ejecución las trazas secuenciales generadas por la función raíz <code>triggerOrchestratorForLead</code>. • Historial del Sistema (System History): Muestra de forma estructurada los registros y eventos que han quedado asentados en las tablas físicas de la base de datos para ese flujo.
--	---

Estado Funcional y Capacidades Técnicas

Integración y Ejecución de Servidor

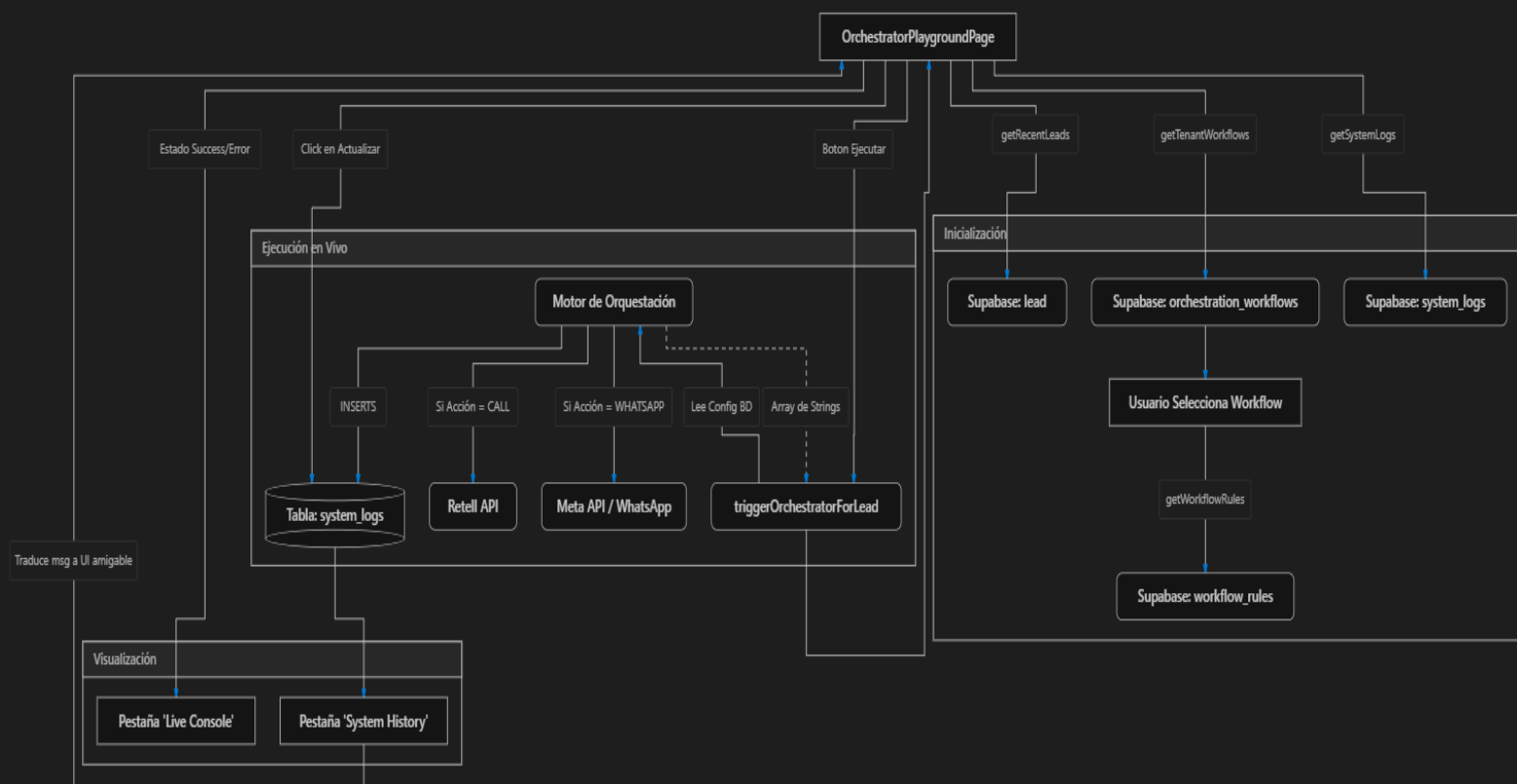
- El módulo se encuentra  **plenamente funcional**. Los mecanismos de extracción de Workflows, Leads y Reglas operan de manera correcta haciendo uso de Server Actions (acciones de servidor nativas de Next.js).
- Invoca directamente al motor real mediante `triggerOrchestratorForLead`, logrando capturar logs de bajo nivel y traduciéndolos a mensajes comprensibles para el administrador frente a fallas comunes (como saldo insuficiente en Meta o inestabilidad en el KB de AWS).

Ajuste de Simulación (Modo Dry-Run)

- **Ejecución Segura en Caliente:** El comportamiento de pruebas ha sido optimizado con éxito. El Playground integra la lógica de control necesaria para recorrer secuencialmente las reglas del workflow e imprimir los logs analíticos en consola **sin disparar peticiones salientes ni llamadas de cobro a APIs de terceros** (Meta Cloud API, Retell AI o infraestructura de AWS).

Conclusión

El Orchestrator Playground se consolida como la consola definitiva de "Test & Debug" de toda la plataforma. Muestra un estado de salud robusto y una arquitectura segura que permite aislar fallos, entender con exactitud por qué un prospecto comercial se estancó en un flujo y auditar el sistema bajo un simulacro controlado y sin costo operativo.



Módulo: Auditoría de Logs de IA

Reporte del Módulo: Auditoría de Logs de la IA

¿Qué es este módulo?

El módulo de **Auditoría de Logs** (/dashboard/logs) funciona como la "caja negra" integral del sistema de orquestación de la plataforma. Está diseñado específicamente para que los ingenieros de soporte, desarrolladores o administradores del sistema puedan inspeccionar en tiempo real cada paso lógico, mensaje de WhatsApp enviado, o intento de llamada iniciado por el motor de IA. Permite diagnosticar con alta precisión si un evento falló, en qué etapa se interrumpió y cuál fue la causa raíz.

Estructura y Componentes Operativos

La interfaz está distribuida de manera síncrona en un diseño de dos paneles complementarios para agilizar el proceso de auditoría:

Sección / Panel	Funcionalidad Técnica y Comportamiento del Sistema
1. Panel Izquierdo (Lista de Eventos)	Actúa como el feed de actividad principal. Ejecuta consultas recurrentes a la tabla orchestration_logs en intervalos de 10 segundos, asegurando una actualización en tiempo real (Poling) sin requerir recargas de página. Despliega de forma compacta el tipo de acción (WHATSAPP, CALL, AI_AGENT), el identificador o nombre del lead afectado, el estado del resultado (SUCCESS / FAILED) y la marca de tiempo exacta. Incluye una barra de búsqueda rápida y selectores de filtrado por estado.
2. Panel Derecho (Inspector de Datos)	Se acciona al seleccionar cualquier registro del feed izquierdo. Funciona como un visor de bajo nivel de metadatos: ● Manejo de Errores: Si el registro contiene un estado fallido, renderiza una alerta visual destacada en color rojo con el mensaje de excepción técnica del proveedor (error_message). ● Visor de Payloads: Expone el objeto JSON original (metadata) dentro de un contenedor con scroll indexado. Este componente es crítico para auditar las estructuras exactas de datos que fueron transmitidas mediante las API salientes hacia Meta Cloud, AWS Bedrock o Retell AI.

Estado Funcional y Limitaciones de Arquitectura

Aspectos Completamente Operativos

- El módulo es  **plenamente funcional**, rápido y ofrece una experiencia de usuario

clara. El mecanismo de auto-refresco automatizado mediante `setInterval` opera de manera limpia y sin fugas de memoria en el cliente.

- Los filtros rápidos y la barra de indexación se procesan directamente sobre la memoria de la interfaz, logrando una respuesta interactiva instantánea.
- El panel de visualización del Inspector realiza el parseo de datos complejos (`JSON.stringify`) de forma correcta, presentando esquemas legibles y organizados para su análisis técnico.

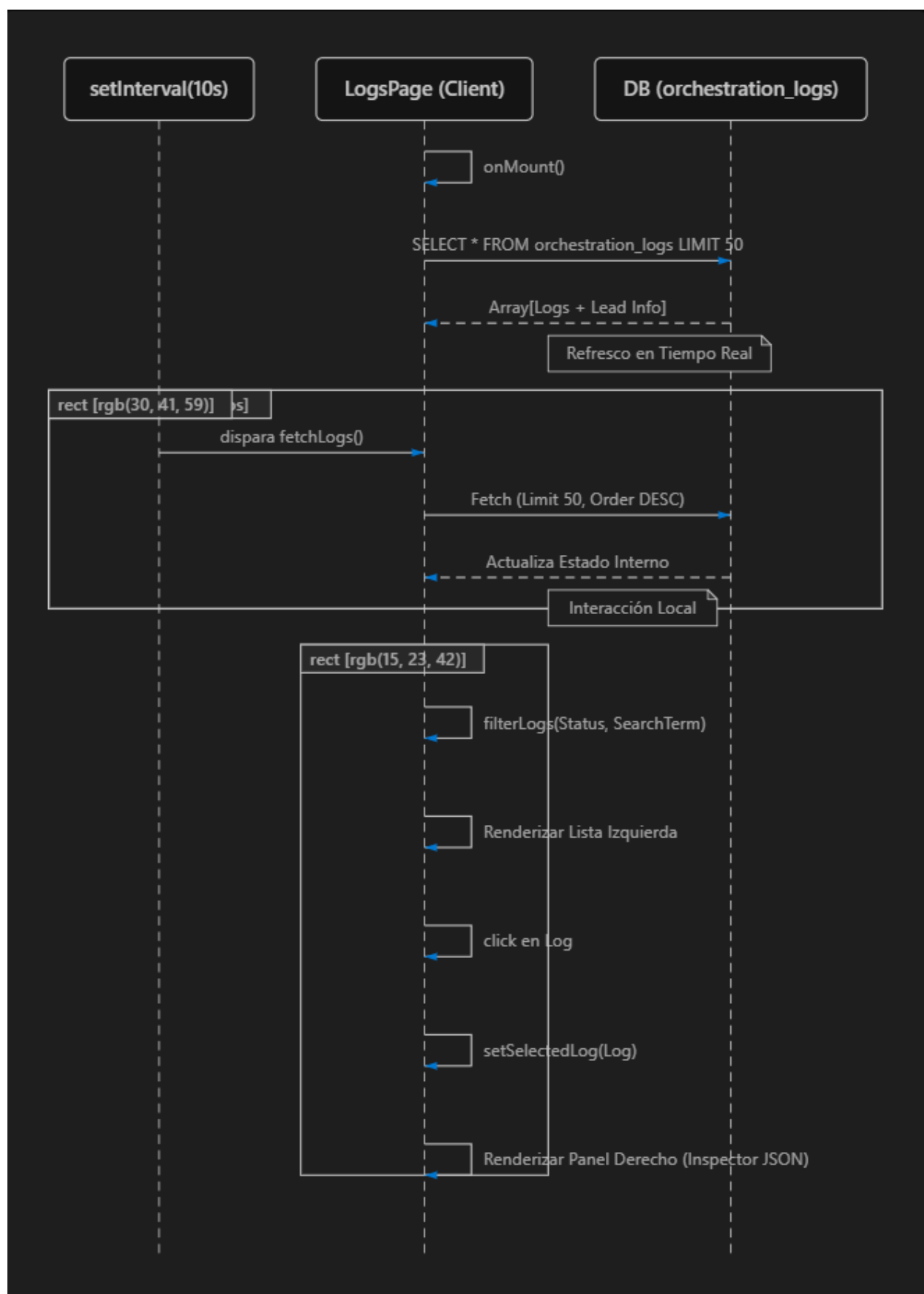
Limitaciones Críticas e Identificación de Brechas (Flaws)

Se detectaron restricciones de diseño en la capa de datos que comprometen la escalabilidad futura del sistema:

- **Restricción de Búsqueda Local (Query Limit):** La consulta actual al backend extrae únicamente un límite fijo de 50 registros (`.limit(50)`). La barra de búsqueda superior aplica filtros exclusivamente sobre este subconjunto de 50 filas locales en el cliente. Si un ingeniero requiere auditar un log específico generado en periodos anteriores (ej. el día anterior), el sistema no lo recuperará a menos que se reestructure mediante una consulta SSR interactiva que ataque directamente a la base de datos con operadores `.ilike()`.
- **Componente UI Incompleto:** El botón interactivo diseñado para la acción de "Copiar JSON" actualmente es un elemento meramente cosmético en el frontend; carece de la lógica de guardado en el portapapeles de la API del navegador (`navigator.clipboard.writeText`).

Conclusión

El panel de Auditoría de Logs de IA es una herramienta de visibilidad de misión crítica para el diagnóstico de excepciones en infraestructuras basadas en IA generativa (donde la volatilidad de tokens de API o timeouts de red son habituales). El módulo cumple con sus objetivos de forma excelente, aunque requiere como prioridad a corto plazo la implementación de paginación real basada en servidor y filtros temporales históricos para soportar auditorías masivas.



Módulo: Simulador de Agentes

Reporte del Módulo: Simulador de Agentes

¿Qué es este módulo?

El **Simulador de Variables** (/dashboard/simulator) es un entorno seguro de pruebas o *Sandbox* diseñado específicamente para evaluar y validar el comportamiento de los Agentes de Inteligencia Artificial (tanto de texto como de voz) antes de habilitar su interacción con leads comerciales reales en producción.

El objetivo fundamental de esta herramienta no se limita a la simple simulación de un chat, sino a auditar de manera estricta y en tiempo real **cómo la IA extrae datos estructurados y variables operativas (memoria de contexto)** a partir del flujo conversacional con el usuario.

Estructura y Flujo de Trabajo

La interfaz del simulador está organizada de forma intuitiva en tres secciones funcionales que guían el proceso de control de calidad (QA):

Sección del Panel	Función Operativa y Comportamiento Técnico
1. Selección de Agente	Ubicado en la sección izquierda del dashboard. Permite al administrador seleccionar de forma dinámica uno de los perfiles de IA configurados y disponibles para el Tenant actual.
2. Chat de Prueba	Ubicado en el área central. Despliega una interfaz de mensajería interactiva donde el evaluador interactúa asumiendo el rol de un "Lead" para poner a prueba los límites del prompt del bot.
3. Monitor de Memoria	Ubicado en el panel derecho. Actúa como un oyente pasivo de la sesión que renderiza visualmente los metadatos y variables que la IA logra parsear y estructurar (por ejemplo, si el usuario escribe <i>"tengo 5 años de experiencia"</i> , el monitor actualiza e ilumina dinámicamente el objeto JSON con la propiedad experiencia: 5).

Estado Funcional y Limitaciones del Entorno (Sandbox)

Aspectos Completamente Operativos

- La interfaz de usuario es altamente reactiva, moderna y asíncrona.
- Mantiene una comunicación bidireccional limpia con la acción de backend `testAgentVariables`, la cual procesa el historial conversacional completo mediante el LLM y retorna de forma síncrona tanto la respuesta en texto plano como el esquema JSON de las variables mapeadas.

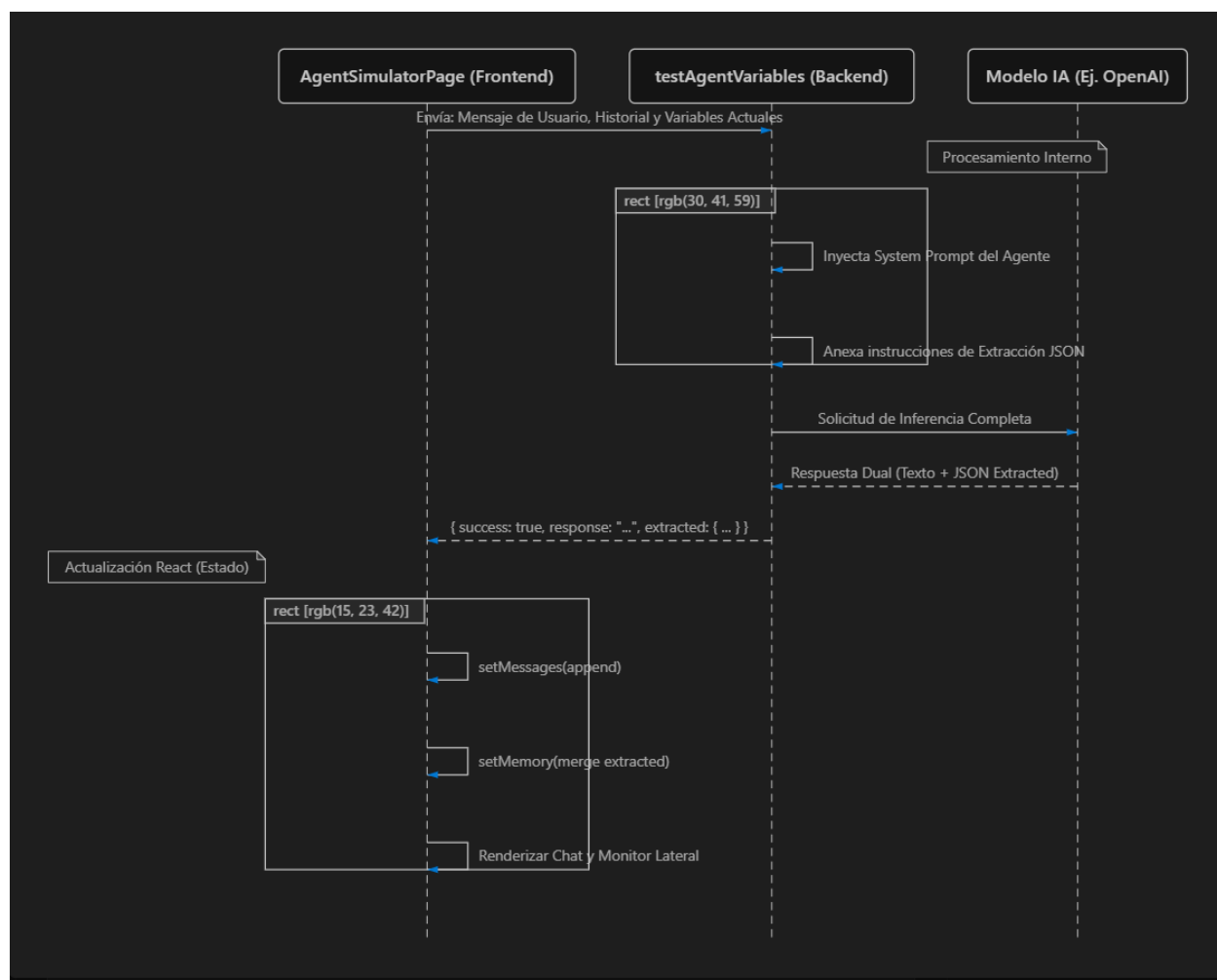
Comportamientos de Diseño y Restricciones del Sistema

Es importante destacar que el módulo no presenta errores de software, sino restricciones explícitas de arquitectura debido a su naturaleza de laboratorio:

- **Persistencia Temporal:** Tal y como se advierte en la interfaz, el guardado de datos opera en modo estricto de Sandbox. Las variables estructuradas bajo este módulo **no persisten en las tablas de Supabase**; solo coexisten en el estado local de la aplicación (`useState`) para fines de demostración y calibración rápida.
- **Volatilidad de la Sesión:** La ventana de chat no retiene el historial de las charlas de prueba si el usuario actualiza, recarga o navega fuera de la página actual.

Conclusión

El simulador constituye una herramienta de validación técnica e ingeniería de prompts impecable para los administradores de la plataforma. Su uso es clave para calibrar, ajustar y auditar el *System Prompt* y las funciones del agente inteligente antes de integrarlo formalmente en campañas de distribución masiva en vivo.



Módulo:Orchestrator Playground

Reporte del Módulo: Orchestrator Playground

¿Qué es este módulo?

El **Orchestrator Playground** (/dashboard/playground) es un entorno de simulación en vivo y diagnóstico avanzado diseñado para que los administradores de la plataforma puedan evaluar el comportamiento de un flujo de trabajo (Workflow) sobre un registro de Lead real seleccionado de la base de datos. Su propósito principal es auditar de manera secuencial y transparente las acciones, ramificaciones y decisiones lógicas que el motor de orquestación central ejecuta paso a paso.

Estructura y Flujo de Operación

La arquitectura del Playground está distribuida estratégicamente para ofrecer un diagnóstico modular e integral:

Componente / Panel	Propósito Operativo y Funcionalidad Técnico
1. Dashboard de Diagnóstico	Se ejecuta de manera automática al cargar el módulo. Realiza un checklist de infraestructura para verificar que el Tenant cuente con las credenciales esenciales activas: Token de Meta/WhatsApp, configuraciones de AWS Bedrock Knowledge Bases (KB) y el estado general de conexión con la Base de Datos.
2. Panel de Setup (Izquierdo)	Concentra los selectores de control del entorno. Permite al usuario elegir un Workflow específico y asociarlo a un Lead real del sistema. Al hacerlo, despliega dinámicamente el listado de reglas y pasos lógicos que componen dicho flujo de automatización.
3. Panel de Ejecución (Derecho)	Monitorea la actividad del proceso mediante dos pestañas de visualización avanzada: ● Consola en Vivo (Live Console): Imprime en tiempo de ejecución las trazas secuenciales generadas por la función raíz triggerOrchestratorForLead. ● Historial del Sistema (System History): Muestra de forma estructurada los registros y eventos que han quedado asentados en las tablas físicas de la base de datos para ese flujo.

Estado Funcional y Capacidades Técnicas

Integración y Ejecución de Servidor

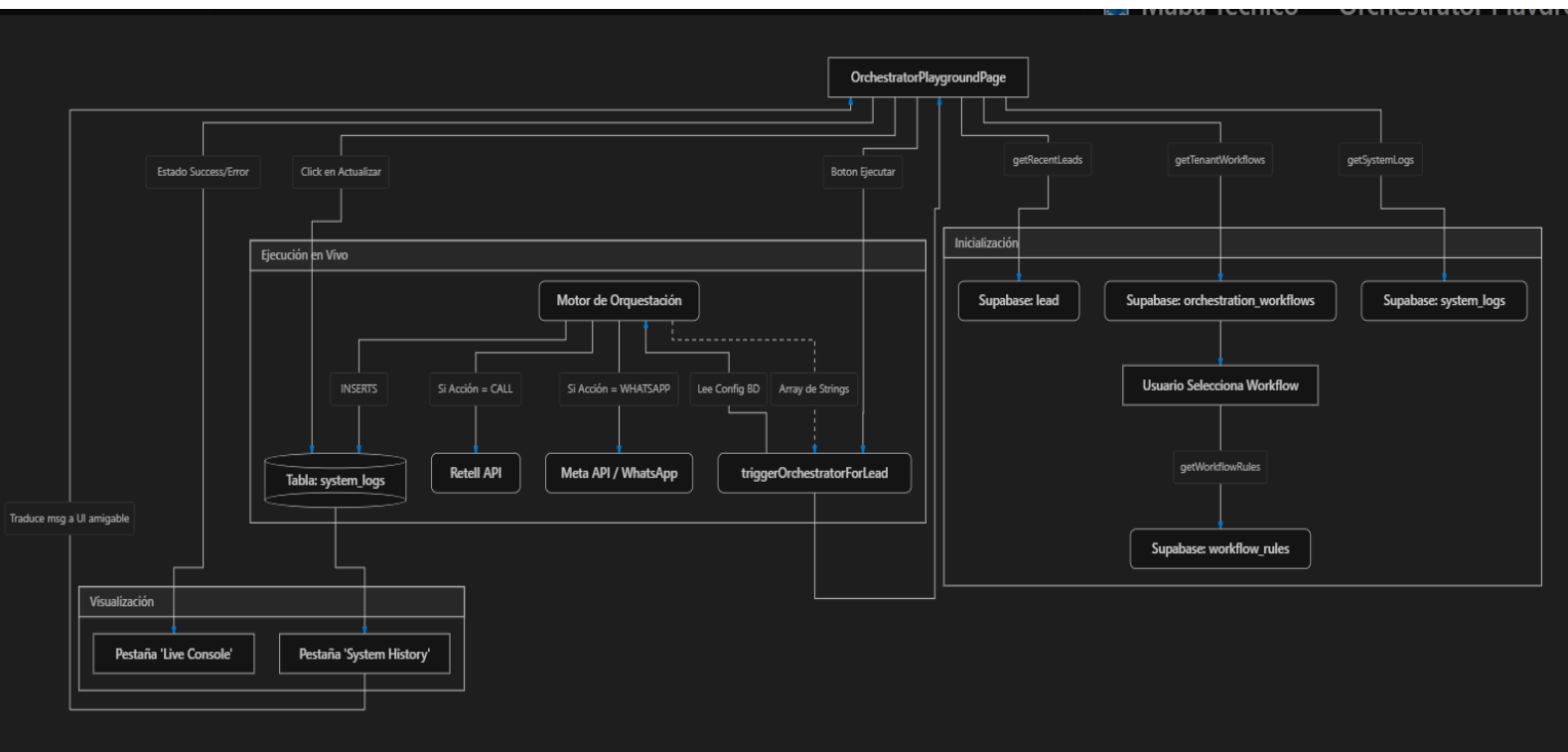
- El módulo se encuentra  **plenamente funcional**. Los mecanismos de extracción de Workflows, Leads y Reglas operan de manera correcta haciendo uso de Server Actions (acciones de servidor nativas de Next.js).
- Invoca directamente al motor real mediante triggerOrchestratorForLead, logrando capturar logs de bajo nivel y traduciéndolos a mensajes comprensibles para el administrador frente a fallas comunes (como saldo insuficiente en Meta o inestabilidad en el KB de AWS).

Ajuste de Simulación (Modo Dry-Run)

- **Ejecución Segura en Caliente:** El comportamiento de pruebas ha sido optimizado con éxito. El Playground integra la lógica de control necesaria para recorrer secuencialmente las reglas del workflow e imprimir los logs analíticos en consola **sin disparar peticiones salientes ni llamadas de cobro a APIs de terceros** (Meta Cloud API, Retell AI o infraestructura de AWS).

Conclusión

El Orchestrator Playground se consolida como la consola definitiva de "Test & Debug" de toda la plataforma. Muestra un estado de salud robusto y una arquitectura segura que permite aislar fallos, entender con exactitud por qué un prospecto comercial se estancó en un flujo y auditar el sistema bajo un simulacro controlado y sin costo operativo.



Módulo: Centro de Costes

Reporte del Módulo: Centro de Costes

¿Qué es este módulo?

El **Centro de Costes** (/dashboard/costs) es el panel de control financiero centralizado del sistema. Está diseñado específicamente para que los administradores de cada Tenant puedan monitorizar y auditar en tiempo real el capital invertido en la operación de la infraestructura de Inteligencia Artificial, desglosado por categorías exactas de gasto. La filosofía técnica y operativa del módulo se rige bajo la premisa: **"Inversión real auditada por lead y categoría."**

Estructura y Fuentes de Gasto

El sistema recopila, calcula de forma independiente y consolida tres fuentes de coste diferenciadas para estructurar la matriz financiera:

Categoría de Gasto	Fuente de Datos (Tablas y Campos)	Tarifa Unitaria Aplicada
Meta API (WhatsApp)	Tabla chat_messages filtrada exclusivamente por registros con dirección de envío saliente (OUTBOUND).	\$0.015 por mensaje enviado.
Modelos de IA	Tabla chat_messages, extrayendo las métricas del campo de metadatos estructurado metadata.token_usage.	\$0.005 por cada 1,000 tokens de entrada. \$0.015 por cada 1,000 tokens de salida.
Telefonía (Voz)	Tabla llamadas, contabilizando los valores registrados bajo la columna duracion_segundos.	\$0.15 por cada minuto consumido.

Lógica de Negocio y Visualización

Cálculo de Inversión por Lead (Top Spenders)

Para identificar los registros con mayor impacto financiero, el motor del backend genera un mapa indexado en memoria (leadMap) que asocia y totaliza los consumos acumulados de cada lead mediante:

- La sumatoria del coste de todos los mensajes de IA y Meta Cloud transmitidos hacia su identificador.
- El coste derivado del tiempo de conversación en llamadas telefónicas procesadas por el bot de voz.

Tras la agregación, el mapa se ordena de forma descendente para renderizar en la interfaz el componente de los **Top 5 Leads** con mayor tasa de inversión asignada.

Gráfico de Flujo de Gastos (Histórico de 14 Días)

El backend genera una serie temporal continua parametrizada para los últimos 14 días. Por cada jornada, el componente de interfaz apila proporcionalmente las tres categorías de gasto (Meta, IA y Voz) en un gráfico de barras acumulativas equipado con tooltips dinámicos que muestran el desglose exacto al pasar el cursor.

Estado Funcional, Hallazgos y Limitaciones (Flaws)

Capacidades Completamente Operativas

- **Precisión en Modelos de IA:** Si el mensaje posee el objeto de telemetría `metadata.token_usage`, procesa los volúmenes reales para fijar el costo de entrada y salida exacto. En caso de ausencia por latencia del proveedor, aplica una tarifa de contingencia base estimada en \$0.002 por mensaje de IA generado.
- **Precisión en Telefonía:** Captura de manera limpia la propiedad `duracion_segundos` de cada registro de llamada, transformando matemáticamente las fracciones a minutos para aplicar el ratio comercial de forma exacta.
- **Gráficos en Vivo:** El panel de barras apiladas se alimenta en su totalidad de datos transaccionales reales extraídos del backend, descartando el uso de mocks visuales.

Limitaciones Críticas de Refinamiento

Se han identificado tres brechas funcionales en el código que requieren intervención técnica antes de catalogar el módulo como producto maduro de producción:

1. **Métrica de Crecimiento Hardcodeada (Growth Metric):** El indicador visual de porcentaje de variación financiera respecto al mes anterior (`growth`) posee un valor estático quemado en el código equivalente a 5.2. No realiza un cálculo comparativo de periodos históricos sobre la base de datos.
`growth: 5.2 // Simulated growth from prev month would need more queries`
2. **Filtro Temporal Inactivo en Base de Datos:** El selector de rangos de la interfaz ("7 Días / 30 Días") altera correctamente el estado de control `timeRange` desencadenando el hook de actualización `useEffect`; sin embargo, las consultas hacia Supabase omiten esta restricción. Las queries descargan el histórico bruto de `chat_messages` y llamadas sin cláusulas de fecha, obligando al panel a mostrar siempre el acumulado histórico total.
3. **Botón de Exportación PDF Huérfano:** El elemento de interfaz diseñado para "Exportar Reporte PDF" se comporta únicamente como un componente cosmético en el frontend. Carece de un controlador de eventos `onClick` o de una biblioteca de renderizado PDF asociada en el cliente o servidor.

Conclusión

El Centro de Costes se presenta como un módulo **analíticamente sólido** que proporciona un cálculo altamente preciso sobre las variables transaccionales base de tokens y minutos de voz empleando datos 100% reales. Sus oportunidades de mejora están acotadas a tres puntos específicos de refactorización: la dinamización del KPI de crecimiento, la incorporación de cláusulas de fecha nativas en las consultas de Supabase y el desarrollo de la función de exportación a PDF.



sistema integral auditoria



Auditoría Integral del Sistema: Automatiza Formación Dashboard

Reporte Consolidado: Seguridad, Lógica de Negocio y Estado Funcional

Elaborado por: Renzo | Sección Analizada: Seguridad, Lógica de Negocio y Estado Funcional

Este informe es el resultado de cruzar tres fuentes de información críticas:

- 1. Auditoría de Seguridad y Calidad (Externa) provista para el proyecto.
- 2. Revisión Funcional de 16 Módulos generada a partir del escaneo directo de la plataforma.
- 3. Validación de Lógica de Negocio comprobada directamente contra el código fuente (lib/core/orchestrator.ts).

1. 🚨 Seguridad y Riesgos Arquitectónicos (Crítico)

La revisión del código confirma las vulnerabilidades detectadas en la auditoría externa. El estado actual de seguridad es **riesgoso** para un entorno de producción.

Riesgo Detectado	Confirmación Técnica	Gravedad
Claves de BD expuestas	Confirmado. En archivos como calls.ts se invoca el cliente de Supabase usando el rol de administrador global (service_role), el cual puentea (bypasses) todas las reglas de seguridad. Si estas claves están expuestas, cualquier persona tiene control total.	🔴 Crítica
Fuga de datos entre Tenants (Clientes)	Confirmado. La separación entre clientes (Tenants) se hace a nivel de "Aplicación" (por variable o cookie), no a nivel de "Base de Datos" (RLS). Modificando una cookie o un parámetro de red, un usuario malintencionado podría ver leads de otra empresa.	🔴 Crítica

Escalada a Administrador	Confirmado. La condición de administrador depende del campo admin o is_admin en los user_metadata de Supabase. Si las reglas RLS de la tabla users no bloquean la actualización, un usuario podría asignarse a sí mismo el rol de Admin.	 Crítica
Páginas abiertas sin autenticación	Confirmado. Rutas API de Webhooks (WhatsApp, Retell, Zoho) carecen de validación de firmas (HMAC) o tokens JWT. El Simulador y el Playground permiten simular llamadas sin control si las rutas no están detrás del Middleware de sesión.	 Alta

2. Lógica de Negocio y Orquestación

La lógica fundacional de negocio fue bien diseñada, pero **falla en su infraestructura de ejecución** y en la estandarización de datos.

2.1 El Flujo de Seguimiento se Congela

- **Expectativa:** El sistema llama el día 1, envía WhatsApp el día 2, etc.
- **Realidad Técnica:** El sistema lee bien la secuencia, pero para ejecutar pasos diferidos envía la tarea a una cola BullMQ. Si el servidor de procesos en segundo plano (Worker) no está encendido o configurado en producción, los pasos 2, 3 y 4 se encolan eternamente pero nunca se ejecutan. El código está bien, la infraestructura no.

2.2 Criterios de Cualificación Incorrectos

- **Expectativa:** Se exige 2 años de experiencia. El sistema debe marcar al lead como "Apto" de forma consistente.
- **Realidad Técnica:** El prompt de la IA lee las reglas dinámicamente (qualification_rules) desde la base de datos. El texto allí probablemente pide 3 años por error humano en la configuración. Además, el estado de cualificación se guarda de forma desordenada (en un lado como SCHEDULING, en otro como qualified: true, y en Zoho con una etiqueta). La falta de un estándar único hace que el embudo se rompa visualmente.

2.3 Lo que sí funciona en el Negocio (Aciertos)

- **Asignación Round Robin:** Funciona y cruza los calendarios correctamente para no asignar citas solapadas.
- **Circuit Breaker (Costos):** El sistema bloquea acciones si el tenant alcanza su límite de gasto diario (daily_spend_limit).
- **Derivación a Humano (Handoff):** Funciona si un número de teléfono es falso o inválido, anulándolo automáticamente en el CRM.

3. Estado Funcional de los Módulos (El Software)

A nivel de software y UI, el dashboard es un panel moderno y robusto. Tras escanear los 16 módulos de la plataforma, el saldo funcional es:

- **80 Funcionalidades 100% Operativas.** (Incluyendo el creador visual de workflows, métricas de costos, A/B testing de agentes, y embudos).
- **8 Funcionalidades con Limitaciones.** (Filtros de fecha que no aplican bien, botones sin acciones conectadas, o métricas "hardcodeadas" como el growth mensual).
- **14 Funcionalidades Faltantes.** (Requerimientos menores aún no desarrollados como envío de archivos adjuntos en Chatbot web, o exportación masiva a CSV).

Veredicto UI/UX: El frontend es altamente profesional, funcional y coherente. El problema del sistema no está en su aspecto ni en la navegación, sino en las capas invisibles (Seguridad y Colas de ejecución).

4. Plan de Acción Unificado (Roadmap Urgente)

Para salvar la producción y asegurar el sistema, se propone el siguiente plan técnico estricto:

Fase 1: Cierre de Brechas (Próximas 48 horas)

- **1. Rotar Claves y Ocultarlas:** Eliminar cualquier clave service_role del código o configuraciones visibles.
- **2. Aislamiento de Clientes (RLS):** Activar Row Level Security en Supabase para que la base de datos fuerce que user.tenant_id === row.tenant_id. Esto mata el Riesgo 2 de raíz.
- **3. Asegurar Webhooks:** Añadir validación de firmas criptográficas a los endpoints de Retell y Meta para evitar inyección de datos falsos.

Fase 2: Restauración de Lógica de Negocio (Próxima semana)

- **1. Reactivar el Seguimiento:** Levantar un servicio dedicado (Worker) para procesar las colas de BullMQ y que la secuencia fluya más allá del Día 1.
- **2. Estandarizar la Cualificación:** Limpiar las reglas de texto en la base de datos (corregir los 2 años) y crear un Enum estricto en el código (Status: QUALIFIED) que actualice todas las tablas a la vez.

Fase 3: Refinamiento Funcional (Próximo mes)

- **1. Resolver Faltantes de UI:** Exportaciones CSV, corregir filtros de fechas en Historial.
- **2. Accesibilidad (WCAG):** Empezar los ajustes de alto contraste y soporte de teclado para cumplir normativas europeas.